DEARFLOW FIELD GUIDE

DearFlow middleware, explained

A visual guide to what happens before and after the model

- This report follows the simplified DearFlow CLI in /Users/johnathanjohnathan/Documents/dearflow. It uses your real ilmu-mini-v3.3 runs, saved before/after snapshots, source code and screenshots of your authenticated LangSmith project.
- All 14 middleware are documented. Each has a purpose, trigger, concrete before/after example and a clear statement of whether it changes the model request, a tool response, graph state or the route through the agent.

- A turn is one user request until DearFlow stops. One turn may contain several model calls, tool calls and subagent calls. A middleware span is one small recorded operation inside that turn.
- Every enabled middleware can run without changing a message. A guard activates only when its condition is met. Title changes a label; memory saving is a side effect; neither rewrites the answer.
- The latest memory learn-and-recall pair passed locally and matched LangSmith snapshots, parent relationships and turn token totals. Older failing memory runs remain failures and are included to explain the fix.
- The original ten-scenario suite exercised all 14 middleware with real model calls. Some guard tests deliberately seed history or force a tool choice. Those fixtures are identified, and the older suite did not complete the same automated remote comparison.

This PDF is the readable companion. The searchable HTML contains complete excerpts, per-field notes and sources, all observed tool arguments, and larger clickable screenshots.

Contents

Contents 2

Start here 3

How a turn flows 6

The real test turns 8

All middleware, one by one 13

Metadata and keywords 27

LangSmith screenshots 42

What the evidence proves 60

Sources and trace links 62

Start here

- Start with How a turn flows, then the turn examples. Read a middleware entry when you want to understand a particular change. Search the HTML glossary for any unfamiliar field.
- In LangSmith, open the DearFlow root run, expand the tree and select middleware.<Class>.<hook>. The prefix middleware. identifies the custom before/after capture. The similar unprefixed span is the framework operation around it.
- For a model wrapper compare Input → before.value with Output → calls[0].request.value. The request is what that wrapper passes onward; later wrappers may still change it. The innermost ChatOpenAI Input shows the final provider-facing message list; tool definitions and model settings are also represented in invocation parameters.
- To inspect the return path compare Output → calls[0].response.value with Output → result.value. For before/after hooks compare Input → before.value with Output → state_update.value; the update is a patch, not the complete new state.
- For tool wrappers compare the requested tool and arguments with the handler response and final result. After a tool returns, its result becomes a message available on the next model call.
- Open Attributes to see Tags, Metadata and Runtime. These describe the run; they are not automatically part of the prompt. Use the glossary to distinguish them from message usage_metadata and tool receipt metadata.
- Screenshots show selected fields, not every line of a large payload. Exact excerpts, run links and the companion evidence files let you inspect the rest. Markdown rendering can display escaped tags as ordinary tags; compare JSON values when checking exact escaping.

What changes versus what merely describes a run

The model sees text in system_message/messages and tool schemas. LangSmith Metadata is the label on the recording. A field called metadata inside an AI or tool message is a different object. The report names the location of each field, because identical words such as id, status and run_id can refer to different things.

One user turn can contain many model calls

- A user turn starts when you submit one request and ends when DearFlow gives its final answer or asks a clarification question. Inside that turn, the model can request tools, read their results, and call the model again. LangSmith shows this work as a parent trace with child runs. A middleware hook is a place where extra code runs before or after part of that work; it is not another user message.
- The verified 03:01 full-flow turn contains seven lead-model calls and four subagent-model calls. The two subagents each call current_time and then write a final report. Their work can overlap; the rows below use recorded span-start order, not a claim that parallel subagent work happened sequentially.
- Before each lead call, the same wrappers run again. Sanitization and PII produce a fresh outgoing copy; the todo wrapper supplies its instruction block; the receipt ledger grows as tool results arrive. Date and memory preparation normally run once at the start of the user turn.

After each model call, the guards inspect its generated tool calls before execution. The final title and memory queue hooks run after the user turn finishes.

- The lead’s seven calls use 34,051 tokens; subagents use 3,277; the root-turn total is 37,328. Reading all wrapper spans as separate model calls would multiply-count the same work. Use real LLM runs and usage_metadata, not the count of middleware spans. Separate memory extraction is outside this root turn.

Call	Actual model output/action	Input tokens	Output tokens	Total tokens	Ledger entries shown
Lead 1	write_todos	3159	136	3295	0
Lead 2	task × 2 (Paris and Berlin)	3558	311	3869	1
Paris • call 1	current_time (Europe/Paris)	586	35	621	0
Berlin • call 1	current_time (Europe/Berlin)	587	36	623	0
Paris • call 2	Final report	756	242	998	1
Berlin • call 2	Final report	755	280	1035	1
Lead 3	write_todos	4579	136	4715	3
Lead 4	calculator: 1 ÷ 0	4920	57	4977	4
Lead 5	write_todos + calculator: 17 × 23	5041	157	5198	5
Lead 6	write_todos	5460	135	5595	7
Lead 7	Final report	5799	603	6402	8

Memory: what failed, what changed, what now passes

- At 02:37 the assistant said it would remember green notebooks and concise bullets, but the first extractor output had malformed JSON. The saved verification review documents two content values missing opening quotes. No facts were saved, so the following full-flow turn had nothing to inject. Later demo instructions were sometimes saved as lasting facts, exposing a separate quality limitation.
- The first repair added one bounded retry and clarified that temporary tool/demo instructions should not become lasting memory. The 03:01 retry still left memory empty after learning. The saved run records show that behavior failure alongside successful remote trace verification; an export check cannot certify the meaning of the model output.
- The current extractor requests a validated MemoryFacts tool schema: a facts list whose entries contain nonempty content, a recognized category and confidence from 0 to 1. Facts below the configured 0.7 threshold are dropped. If output is malformed, one retry is allowed. This schema helps enforce shape; it does not prove that extracted facts are true or useful.
- The 03:30 learning turn saved both expected preferences on its first extraction attempt. A separate new thread then asked for the preferences without providing the answers. Its before_agent snapshot contains both saved facts in <memory>, and its final answer correctly

recalls both. The two chat turns and two extraction calls total 7,502 tokens: 3,092 + 626 + 3,146 + 638.

- All 36 custom middleware snapshots across that pair matched LangSmith, with matching parent relationships and root-turn usage. The extraction diagnostic records show one completed MemoryFacts call for each turn. A duplicate paraphrase of the green-notebook fact remains after recall; this passing smoke test does not demonstrate general memory accuracy, semantic deduplication, or every provider’s behavior.

What these demonstrations prove - and what they leave open

- All 14 middleware classes appear in the original broad run set. That is coverage of the components, not proof that every optional branch ran. Some hooks legitimately make no change. The detailed component pages distinguish observed examples from additional behavior established by the current code.
- The long-history, damaged-history, repeated-call, delegation-cap and pending-todo cases use explicit fixtures. The continuation calls are real provider calls, but their starting histories or tool-choice constraints are deliberately constructed. Threshold overrides are identified beside each scenario so they are not confused with normal settings.
- Historical runs retain their historical status. The original broad suite has eight passing local scenarios and two memory failures; its old remote readback verifier failed. The later 03:01 pair has exact remote matches but still failed memory behavior. The 03:30 memory pair passes both behavior and remote checks. The report combines these evidence sets rather than claiming a single all-branches successful run.
- Middleware is control and context management, not a truth guarantee. The older time examples contained incorrect weekday/date prose even though current_time returned a correct timestamp, and receipt labels can be misinterpreted across lead and subagent ledgers. A receipt confirms a call happened; a todo marked completed records a model-maintained status; a schema validates structure. Important factual claims still need checking against the actual tool result.

How a turn flows

1. Start the turn

DynamicContext adds the date; Memory adds saved user facts if available. The state holds messages and working fields such as todos.

2. Prepare the next model request

Summarization can shorten old history. Todo can add a reminder. Wrappers add receipts, sanitize user text, repair tool history, warn about budget or loops, add planning instructions and mask PII.

3. Call the model

The model receives the assembled system message, messages and tool definitions. It returns text, requested tool calls and token usage. A requested tool call is not yet an executed action.

4. Apply response guards

Clarification, Todo, LoopDetection, TokenBudget and SubagentLimit inspect the returned message in reverse middleware order. They may preserve it, add a warning, remove tool calls or change the next step.

5. Execute tools or finish

If tools remain, their wrappers handle errors, clarification formatting and execution receipts. The loop returns to another model call. Otherwise Title labels the thread and Memory queues extraction for later turns.

Which hooks run at each stage

Before agent: DynamicContext, Memory.

Before the model: RedactingSummarization, Todo.

Wrap model call outer to inner: ToolReceipt, InputSanitization, DanglingToolCall, TokenBudget, LoopDetection, Todo, PiiRedaction.

After the model: Clarification, Todo, LoopDetection, TokenBudget, SubagentLimit.

Wrap tool call outer to inner: ToolReceipt, ToolErrorHandling, PiiRedaction, Clarification.

After agent: Title, Memory.

Current nonsecret settings

plan_mode: Yes

context: timezone=None; date_reminder=Yes; summarization.enabled=Yes; summarization.trigger_tokens=100000; summarization.keep_messages=20; summarization.model=None; title.enabled=Yes; title.use_model=No; title.max_words=6

memory: enabled=Yes; storage_path=memory; injection_enabled=Yes; debounce_seconds=10; max_facts=100; confidence_threshold=0.7; max_injection_tokens=1500; model=None

subagents: enabled=Yes; max_concurrent=3; max_total=50; max_turns=25; timeout_seconds=300

token_budget: enabled=Yes; max_tokens=200000; max_input_tokens=None; max_output_tokens=None; warn_threshold=0.8; hard_stop_threshold=1.0

loop_detection: enabled=Yes; warn_threshold=3; hard_limit=5; window_size=20

verification: receipts_enabled=Yes; receipts_render_mode=always

observability: middleware_snapshots=Yes; capture_content=Yes; max_snapshot_chars=100000

pii_redaction: enabled=Yes; redact_email=Yes; redact_api_key=Yes; redact_credit_card=Yes; redact_phone=Yes; redact_national_id=Yes; tool_names=[]

Subagent scope

Subagents start a fresh context with the task prompt and filtered tools, and inherit only this smaller middleware set. They do not automatically receive the lead conversation, date, long-term memory, summaries, titles or todo plan. Their token usage still contributes to the shared user-turn budget.

Subagent middleware: ToolReceiptMiddleware, ToolErrorHandlingMiddleware, InputSanitizationMiddleware, DanglingToolCallMiddleware, TokenBudgetMiddleware, LoopDetectionMiddleware, PiiRedactionMiddleware

The real test turns

01. 02:37 UTC batch • memory-learn

Result: FAIL / incomplete - local evidence; original automatic LangSmith comparison not verified

Before: The synthetic user states two lasting preferences: green notebooks and concise bullet points. There are no saved preferences to inject.

After: Date context and a title were added. The assistant acknowledged both preferences, but memory persistence failed. The acknowledgement alone did not mean anything was saved.

Token usage: 3,097 total = 3,035 input + 62 output; 1 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Ordinary live request; no seeded history or forced tool choice.

02. 02:37 UTC batch • full-flow

Result: FAIL / incomplete - local evidence; original automatic LangSmith comparison not verified

Before: A new request asks for two city-time subagents, a deliberately failing division, a multiplication and a completed todo list. The previous learning turn left no saved preferences.

After: Reserved tags were escaped, the fictional email was masked, and a date reminder was supplied. Two subagents returned real time results; division by zero became a recoverable error; multiplication returned 391.0. All five todos ended completed and the lead collected nine receipts. The memory-injection check failed because the previous turn saved no preferences.

Token usage: 34,386 total = 31,938 input + 2,448 output; 10 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Ordinary live request; no seeded history or forced tool choice.

03. 02:37 UTC batch • clarification

Result: PASS - local evidence; original automatic LangSmith comparison not verified

Before: The user asks the model to ask a Tokyo-or-Sydney clarification question and perform no other task.

After: The model requested ask_clarification. Middleware intercepted it without invoking the inner tool handler, formatted the two choices, and ended the turn. There was only one model call. The run did not exercise sibling-tool removal because no sibling call was produced.

Token usage: 3,227 total = 3,144 input + 83 output; 1 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Ordinary live request; no seeded history or forced tool choice.

04. 02:37 UTC batch • interrupted-history

Result: PASS - local evidence; original automatic LangSmith comparison not verified

Before: The fixture supplies one prior current_time call with no result and an orphan calculator result. Neither seeded item is claimed as a live execution.

After: The outgoing request gains an interrupted-call error placeholder and loses the orphan result. A real continuation model and a fresh Europe/Paris current_time call completed. The final prose incorrectly said Friday for 2026-10-05, illustrating that valid middleware flow does not guarantee factual wording.

Token usage: 6,870 total = 6,748 input + 122 output; 2 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Seeded missing tool response and orphan result; continuation and current-time lookup use the real provider/tools.

05. 02:37 UTC batch • summarization

Result: PASS - local evidence; original automatic LangSmith comparison not verified

Before: Eight fictional workshop exchanges are seeded, and the demo lowers the summary trigger to 800 tokens while keeping four recent messages.

After: The middleware called a real summary model, redacted the fictional contact, and replaced older history with a summary plus retained recent messages. The lead correctly returned Cedar Workshop, 30 minutes and green notebooks. Of the 6,760 recorded tokens, 3,025 belonged to the summarization model and 3,735 to the lead.

Token usage: 6,760 total = 6,363 input + 397 output; 2 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Seeded fictional workshop history. Demo-only summary threshold 800 tokens, keep 4 messages; summary and final answer use real model calls.

06. 02:37 UTC batch • loop-limit

Result: PASS - local evidence; original automatic LangSmith comparison not verified

Before: Three completed, identical calculator calls are seeded. The provider is instructed to keep choosing calculator; the actual model supplies the arguments.

After: The next model request receives LOOP DETECTED. The fourth identical call executes; the fifth generated request is stripped of its tool call and replaced with FORCED STOP. stop_reason is loop_capped. This is a deliberate stress fixture, not an observed natural model loop.

Token usage: 6,450 total = 6,403 input + 47 output; 2 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Three seeded identical completed calculator calls. Provider tool_choice forces calculator on real continuation calls; model generates arguments. Production loop thresholds remain 3/5.

07. 02:37 UTC batch • delegation-limit

Result: PASS - local evidence; original automatic LangSmith comparison not verified

Before: Three completed task calls are seeded and the demo reduces max_total to 3. The next real model response is forced to request task.

After: SubagentLimit removes the newly generated task call before execution and adds SUBAGENT LIMIT REACHED. It proves enforcement of the test cap; it does not launch another subagent. Normal configuration still permits 50 total task calls per user turn.

Token usage: 3,287 total = 3,252 input + 35 output; 1 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Three seeded completed delegations; demo-only max_total=3. Real next model response is forced to request task and middleware must remove it before execution.

08. 02:37 UTC batch • todo-reminder

Result: PASS - local evidence; original automatic LangSmith comparison not verified

Before: The fixture stores a pending todo but omits its original write_todos history. The provider is set to tool_choice=none so it produces final-text answers.

After: A hidden reminder restores the pending plan. Two after-model reminders route the graph back to the model. After three model calls, the bounded reminder guard stops, with the todo intentionally still pending. PASS means the restoration and reminder behavior happened, not that this task was completed.

Token usage: 10,767 total = 10,761 input + 6 output; 3 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Seeded pending todo with missing original tool history; provider tool_choice=none forces real final-text responses. The guard reminds twice, then stops with todo still pending by design.

09. 02:37 UTC batch • budget-warning

Result: PASS - local evidence; original automatic LangSmith comparison not verified

Before: A normal Europe/Paris time request runs with a demo-only warning threshold of 0.1% of the 200,000-token budget.

After: After 3,515 real tokens, the next request receives TOKEN BUDGET WARNING. The turn finishes after two model calls, totaling 7,253 tokens, without a hard stop. Its final prose wrongly states Tuesday, October 6 despite the tool returning October 5; the middleware warning test is separate from factual answer quality.

Token usage: 7,253 total = 7,198 input + 55 output; 2 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Ordinary live tool request. Demo-only soft warning at 0.1% of the 200000-token cap; actual provider usage triggers it.

10. 02:37 UTC batch • budget-stop

Result: PASS - local evidence; original automatic LangSmith comparison not verified

Before: A normal multiply-17-by-23 request runs with an intentionally tiny 1,000-token hard cap.

After: The first model response takes accumulated usage to 3,530 tokens. Middleware removes its calculator call before execution, appends TOKEN BUDGET EXCEEDED and records token_capped. This turn did not calculate 391 through the tool; it demonstrates an after-response hard stop and the possibility of overshooting a cap within one model call.

Token usage: 3,530 total = 3,491 input + 39 output; 1 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Ordinary live request with a demo-only 1000-token budget; real provider token usage must trigger the stop.

11. 03:01 UTC batch • memory-learn

Result: FAIL / incomplete - exact LangSmith comparison verified

Before: The memory retry again begins with a fresh synthetic user and the same two explicit preferences.

After: The assistant acknowledges the preferences, but memory_before and memory_after remain empty. Both memory_persisted and memory_preferences_persisted fail. LangSmith export succeeds: snapshots, parents and 3,079 root-turn tokens match. A correctly exported failed run is still a failed behavior test.

Token usage: 3,079 total = 3,035 input + 44 output; 1 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Ordinary live request; no seeded history or forced tool choice.

12. 03:01 UTC batch • full-flow

Result: FAIL / incomplete - exact LangSmith comparison verified

Before: The follow-up full-flow request starts with no memory because learning failed again.

After: Sanitization, PII masking, date context, tool-error recovery, receipts, two delegated time jobs and all five todos work. The lead collects eight receipts. The two preference-injection checks fail. Some unrelated demo facts were saved after this turn, which does not rescue the missing

before-turn preferences. LangSmith matches the recorded snapshots, parents and 37,328 turn tokens.

Token usage: 37,328 total = 35,200 input + 2,128 output; 11 model call(s) inside the user-turn trace. Separate background memory extraction is not included in this turn total.

Open this turn in LangSmith

Test setup: Ordinary live request; no seeded history or forced tool choice.

13. 03:30 UTC batch • memory-learn

Result: PASS - exact LangSmith comparison verified

Before: A fresh synthetic user explicitly supplies the two preferences. Chat tools are disabled by provider tool_choice=none; the separate extractor is allowed its MemoryFacts schema.

After: The latest extractor returns valid structured MemoryFacts on the first attempt. Both preferences are saved and the local learning checks pass. The chat turn costs 3,092 tokens; extraction adds 626. All 18 custom snapshots match LangSmith, alongside parent links and the root-turn token total.

Token usage: 3,092 total = 3,035 input + 57 output; 1 model call(s) inside the user-turn trace. Separate memory extraction: 626 tokens; combined 3,718.

Open this turn in LangSmith

Test setup: Synthetic preference learning; provider tool_choice=none prevents chat tools, while memory extraction binds its own facts schema.

14. 03:30 UTC batch • memory-recall

Result: PASS - exact LangSmith comparison verified

Before: A fresh thread for the same synthetic user asks what notebook color and answer style they prefer, without supplying either answer.

After: MemoryMiddleware injects the saved green-notebook and concise-bullet preferences. The assistant correctly recalls both. The chat turn costs 3,146 tokens; first-attempt extraction adds 638. All 18 custom snapshots match LangSmith. The saved store contains a paraphrase duplicate of the green preference after recall, so semantic deduplication is not established.

Token usage: 3,146 total = 3,071 input + 75 output; 1 model call(s) inside the user-turn trace. Separate memory extraction: 638 tokens; combined 3,784.

Open this turn in LangSmith

Test setup: Fresh thread for the same synthetic user; the question omits both answers. Provider tool_choice=none prevents chat tools. Saved memory must be injected and recalled.

All middleware, one by one

01. ToolReceiptMiddleware

Keeps an execution record so the model can point to tools that actually ran. A receipt says what ran and whether it succeeded; it does not prove an answer is correct.

Hooks: wrap_model_call, wrap_tool_call

Screenshots: Figure 4 (Receipts: the model receives an execution record).

Before the model

Appends a hidden receipt ledger to the model request when receipts exist. In the current demo render_mode is always. The other mode, delegation_only, waits until a subagent result is present in the current turn.

After the model

When the wrapped model returns, stamps the exact ledger shown into the AI message as dearflow_tool_receipt_ledger. It does not rewrite the model’s answer text.

Around tools

After each tool finishes, stamps dearflow_tool_receipt into its ToolMessage, including errors returned by inner middleware. The receipt layer is outermost so it sees the final tool result.

When nothing changes

No ledger text is added if there are no receipts. A reply can still carry an empty stamped ledger [].

Limits and cautions

- Receipt ids such as r1 are local to the ledger shown to that agent; a subagent’s r1 need not be the lead agent’s r1.
- Hashes are compact fingerprints, not full tool arguments or outputs.
- A citation resolving to a successful receipt confirms execution, not correctness or user acceptance.

Observed example 1

Test turn: full-flow

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: No receipt-ledger message at the end of this request.

After: ## Tool receipts (execution record)
Cite receipt ids (e.g. [r1 write_file]) in your final report for every claim about an action you took.
Execution evidence only: receipts record that a call happened and its status; they do not validate claim correctness or task acceptance.
- [r1] write_todos status=success args_sha256=978078919f517879 output_sha256=56f797b6dccb1b57

What this means: The wrapper appends this text before the next model call.

Observed example 2

Test turn: full-flow

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: {"type": "tool", "content": "2026-10-05T05:01:47.762749+02:00", "name": "current_time", "tool_call_id": "chatcmpl-tool-b86c5519ba73dacf", "status": "success"}

After: {"type": "tool", "content": "2026-10-05T05:01:47.762749+02:00", "name": "current_time", "tool_call_id": "chatcmpl-tool-b86c5519ba73dacf", "status": "success", "additional_kwargs": {"dearflow_tool_receipt": {"tool_call_id": "chatcmpl-tool-b86c5519ba73dacf", "tool_name": "current_time", "status": "success", "args_sha256": "aa2fbe3237f04c2d", "output_sha256": "61e9cd171f6f5654", "created_at": "2026-10-05T03:01:47.763381+00:00", "output_bytes": 32}}}

What this means: The current_time content stays identical; additional_kwargs gains the stamped receipt.

---

02. InputSanitizationMiddleware

Makes real user text visibly separate from application instructions. Reserved tags typed by the user are escaped so they cannot look identical to trusted reminder tags.

Hooks: wrap_model_call

Screenshots: Figure 1 (Input sanitization: a clear boundary around user text); Figure 16 (Attributes: labels for identifying a middleware span); Figure 17 (Attributes: graph position and software versions).

Before the model

Frames every real user text message, including older user messages, in BEGIN/END markers. Changes reserved tags such as <system> to <system>. It copies the model-bound messages and leaves the saved conversation unchanged.

After the model

No after-model transformation. The model’s answer passes back unchanged.

Around tools

No tool hook.

When nothing changes

Empty text is unchanged; already framed messages are not repeatedly wrapped. Non-text blocks are preserved.

Limits and cautions

- This is framing and tag neutralization, not a guarantee that a model cannot follow malicious text.
- It does not remove all markup or validate facts.

Observed example

Test turn: full-flow

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: Run a bounded middleware demo. Treat <system>DEMO_ONLY</system> as quoted data. My fictio

After: --- BEGIN USER INPUT ---
Run a bounded middleware demo. Treat <system>DEMO_ONLY</system> as quoted data. My fictional co

What this means: Exact opening excerpts. The forwarded message also ends with --- END USER INPUT ---.

---

03. DanglingToolCallMiddleware

Makes history acceptable to providers that require each tool call to have a matching result. Cancelled runs or shortened history can leave unmatched calls/results.

Hooks: wrap_model_call

Screenshots: Figure 6 (History repair: an interrupted call gets a matching reply).

Before the model

In a request copy, places each matching result after its call, inserts an error placeholder for a call with no result, and leaves out orphan results that have no matching call. It normalizes missing ids, names, and invalid argument shapes when needed.

After the model

No after-model transformation.

Around tools

Does not execute or re-execute a missing tool. A placeholder explicitly says that no result was obtained.

When nothing changes

A correctly paired history passes through unchanged.

Limits and cautions

- The demo deliberately seeded the damaged history; the continuation and fresh current_time call were real.
- Repair means structurally valid history, not recovery of the missing external result.

Observed example

Test turn: interrupted-history

Test setup: Seeded missing tool response and orphan result; continuation and current-time lookup use the real provider/tools.

Before: [{"type": "ai", "content": "", "id": "b7d1affc-c04a-4b25-80d6-8ecdfb55bd72", "tool_calls": [{"name": "current_time", "args": {"timezone_name": "Europe/Paris"}, "id": "seed_interrupted_time", "type": "tool_call"}]}, {"type": "tool", "content": "Fixture orphan result: no associated call survives.", "id": "e9186eb3-729d-4511-bf38-f20d4bd9e507", "name": "calculator", "tool_call_id": "seed_orphan_call", "status": "success"}]

After: [{"type": "ai", "content": "", "id": "b7d1affc-c04a-4b25-80d6-8ecdfb55bd72", "tool_calls": [{"name": "current_time", "args": {"timezone_name": "Europe/Paris"}, "id": "seed_interrupted_time", "type": "tool_call"}]}, {"type": "tool", "content": "[Tool call was interrupted and did not return a result.]", "name": "current_time", "tool_call_id": "seed_interrupted_time", "status": "error"}]

What this means: The orphan calculator result disappears; the interrupted current_time call receives an error placeholder.

04. ToolErrorHandlingMiddleware

Lets the agent continue after an ordinary tool failure instead of crashing the whole user turn.

Hooks: wrap_tool_call

Screenshots: Figure 5 (Tool errors: an exception becomes a model-readable result).

Before the model

No direct model-request edit. A recovered error becomes a ToolMessage in history and is visible to the next model call.

After the model

No edit to the model’s answer itself.

Around tools

Catches ordinary exceptions from the inner tool handler and returns status=error with the tool name and failure text. LangGraph control-flow exceptions are re-raised.

When nothing changes

Successful tool results pass through unchanged.

Limits and cautions

- The demo proves recovery from calculator division by zero; it does not prove every possible external tool failure.
- Recovery does not make the failed operation succeed.

Observed example

Test turn: full-flow

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: {"name": "calculator", "args": {"left": 1, "right": 0, "operation": "divide"}, "id": "chatcmpl-tool-ab5f29261fd44dae", "type": "tool_call"}

After: {"type": "tool", "content": "Tool 'calculator' failed: Cannot divide by zero", "tool_call_id": "chatcmpl-tool-ab5f29261fd44dae", "status": "error"}

What this means: calculator(1, 0, divide) becomes “Tool 'calculator' failed: Cannot divide by zero”. The following multiplication succeeded.

05. DynamicContextMiddleware

Gives the model today’s date without changing the stable base system prompt.

Hooks: before_agent

Screenshots: Figure 2 (Dynamic context: the date becomes a hidden message).

Before the model

Adds a hidden <current_date> message around the latest user message at the start of a turn, when a date reminder is absent or out of date.

After the model

No after-model edit.

Around tools

No tool hook; it reads the application clock rather than calling the model.

When nothing changes

If the conversation already contains a reminder for the same day, it adds nothing.

Limits and cautions

- A correct date reminder does not guarantee the model’s prose uses the right weekday. One older time-test answer incorrectly said Friday despite the Monday reminder.
- Timezone is the configured IANA zone or the host local zone.

Observed example

Test turn: full-flow

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: The state contains the user message with no date reminder.

After: <system-reminder>
<current_date>2026-10-05, Monday</current_date>
</system-reminder>

What this means: This is the added hidden message, not a rewrite of the base system prompt.

---

06. MemoryMiddleware

Carries useful facts from one conversation into a fresh conversation for the same user.

Hooks: before_agent, after_agent

Screenshots: Figure 3 (Memory: saved preferences enter a new thread); Figure 18 (Historical memory failure: the model returned malformed JSON).

Before the model

Loads saved facts and inserts a hidden <memory> message before the user prompt. A digest prevents adding the same text repeatedly.

After the model

After the whole agent turn finishes, queues the latest real user message and final assistant text for background extraction. This hook returns null even when it successfully queued work. Persistence happens later; the demo flushes the queue and checks the file.

Around tools

Tool calls and hidden reminder text are excluded from the extracted exchange.

When nothing changes

No injection when memory is empty, disabled for injection, or unchanged; no exchange is queued when there is no non-empty real user message.

Limits and cautions

- Earlier 02:37 and 03:01 learning attempts failed. The later 03:30 smoke test is the passing evidence for current structured MemoryFacts extraction.
- A model saying “I will remember” is insufficient; saved facts and fresh-thread recall were checked.
- Memory extraction runs outside the completed turn trace in this demo and its tokens are counted separately.

- Recall produced a paraphrase duplicate of the green-notebook preference in stored memory, so this test is not proof of semantic deduplication.

Observed example 1

Test turn: memory-recall

Test setup: Fresh thread for the same synthetic user; the question omits both answers. Provider tool_choice=none prevents chat tools. Saved memory must be injected and recalled.

Before: What notebook color and answer style do I prefer? Use what you remember about me. Answer briefly without tools.

After: <memory>
What you know about this user:
- [preference] Prefers concise bullet points for information presentation.
- [preference] Favors green as their favorite notebook color.
</memory>

What this means: The question itself omits green and concise bullets. Saved memory supplies both, and the fresh-thread final answer recalls both.

Observed example 2

Test turn: memory-learn

Test setup: Synthetic preference learning; provider tool_choice=none prevents chat tools, while memory extraction binds its own facts schema.

Before: Final user/assistant exchange is available.

After: None

What this means: state_update=null is expected: this hook queues a side effect. Evidence of success is memory_after containing both saved preferences, plus the later recall.

---

07. RedactingSummarizationMiddleware

Keeps long conversations within a manageable context size while retaining a concise account of earlier work.

Hooks: before_model

Screenshots: Figure 7 (Summarization: replace older messages with a summary).

Before the model

When estimated message tokens reach the trigger, first redacts personal data in the material sent to a separate summary model. Replaces older messages with the generated summary and retains recent messages. It inherits the ordinary summarization behavior and adds redaction for this otherwise separate model path.

After the model

No direct change to the lead model’s answer. The separate summary model’s output is used as new context before the lead model runs.

Around tools

No tool hook.

When nothing changes

Below the summarization trigger it returns no state update.

Limits and cautions

- The demonstration used fictional seeded workshop history, an 800-token trigger and a 4-message keep setting. It was not a naturally accumulated long chat.
- Summaries can lose details; retention counts can be adjusted for valid message boundaries.
- The separate summary model bypasses the main PII wrapper, which is why this redacting subclass exists.

Observed example

Test turn: summarization

Test setup: Seeded fictional workshop history. Demo-only summary threshold 800 tokens, keep 4 messages; summary and final answer use real model calls.

Before: {"message_count": 19, "first_message_excerpt": "Seeded workshop planning record 1: the budget discussion concerned a fictional educational event called Cedar Workshop. The agreed outcome is a small practical session with clear examples, accessible explanations, time for questions, and co"}

After: {"state_update_message_count": 6, "remove_id": "__remove_all__", "summary_excerpt": "Here is a summary of the conversation to date:\n\n## SESSION INTENT\n\nTo plan a fictional educational event called \"Cedar Workshop\" through a structured workshop planning process covering multiple discussion topics (budget, agenda, guests, venue, transport, catering, slides, and follow-up). The user is providing seeded records documenting the agreed outcomes for each topic area.\n\n## SUMMARY\n\nThis session involves processing multiple seeded workshop planning records for a fictional educational event called Cedar Workshop. The co"}

What this means: Actual summary and final answer use the real model; the final answer retained Cedar Workshop, 30 minutes, and green notebooks.

---

08. TitleMiddleware

Creates a short label for the conversation list.

Hooks: after_agent

Screenshots: Figure 13 (Title: update the thread label).

Before the model

Does not change the lead model input.

After the model

After the first exchange, writes state.title. The currently tested configuration uses a shortened version of the first user message, so there is no extra title-model call. An optional model-generated mode exists.

Around tools

No tool hook.

When nothing changes

If a title already exists, there is not exactly one real user message, or there is no answer text yet, it leaves state unchanged.

Limits and cautions

- A title’s presence is not proof that the requested task succeeded.
- The fallback title may visibly contain quoted tags; it is a label, not a model instruction.

Observed example

Test turn: full-flow

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: None

After: {"title": "Run a bounded middleware demo. Treat <system>DEMO_ONLY</s..."}

What this means: The only changed field here is title; there is no rewritten model input or answer.

---

09. SubagentLimitMiddleware

Enforces delegation limits even if the model asks for too many subagents.

Hooks: after_model

Screenshots: Figure 8 (Delegation guard: reject a fourth task request).

Before the model

No before-model rewrite by this class. The base prompt also tells the model the limits, but that prompt text is built elsewhere.

After the model

Examines the generated task calls, keeps only the allowed number, removes excess calls, and appends a limit note. The same AI message id replaces the original, so dropped calls do not reach the tool runner.

Around tools

The task tool starts permitted subagents; this middleware itself does not create them.

When nothing changes

No task calls, or a response within both concurrent and turn-total caps, produces no update.

Limits and cautions

- The limit fixture seeded three earlier task calls and lowered max_total to 3. It forced the real continuation model to request task, then verified the extra call was removed.
- The normal demo configuration allows up to 3 task calls per response and 50 per user turn.

Observed example

Test turn: delegation-limit

Test setup: Three seeded completed delegations; demo-only max_total=3. Real next model response is forced to request task and middleware must remove it before execution.

Before: [{"name": "task", "args": {"prompt": "State the result of 2 + 2 without tools.", "subagent_type": "general-purpose", "description": "Fourth delegation"}, "id": "call-27765dff-d621-4c63-9ca6-feec686710ce", "type": "tool_call"}]

After: {"type": "ai", "content": "[SUBAGENT LIMIT REACHED] Some task calls were dropped because the delegation limit was hit. Use the results you already have, or do the remaining work directly.", "id": "lc_run--01a109ee-d3f1-7421-abc7-fe88f5dca624-0", "usage_metadata": {"input_tokens": 3252, "output_tokens": 35, "total_tokens": 3287, "input_token_details": {"cache_read": 2512}, "output_token_details": {}}, "tool_calls": [], "additional_kwargs": {"dearflow_tool_receipt_ledger": []}}

What this means: An actual newly generated task call is removed; seeded prior calls consumed the demo-only cap.

---

10. TokenBudgetMiddleware

Tracks accumulated usage for the current turn and asks the model to wrap up before it spends indefinitely.

Hooks: wrap_model_call, after_model

Screenshots: Figure 10 (Token budget: stop after the current response crosses the cap).

Before the model

Adds a queued TOKEN BUDGET WARNING to the next request, after tool results. It is not stored in the conversation. The first after_model check sets the warning flag; the next wrapper delivers it.

After the model

At the hard threshold, removes all tool calls from the generated answer, adds a budget-exceeded note, and marks stop_reason=token_capped.

Around tools

Budget counts lead-agent, subagent and middleware model calls tracked inside the run; the same guard is inherited by subagents.

When nothing changes

Below thresholds, or outside a tracked RunService turn, it leaves the messages unchanged.

Limits and cautions

- The hard cap is checked after a model response, so a single response can overshoot the limit. It is not a provider-side prepaid cutoff.
- Separate background memory extraction in this demo is outside the root turn budget and is reported separately.
- The warning demo intentionally used a 0.1% threshold; the stop demo used a 1,000-token cap. These are test overrides, not the normal 200,000-token cap.

Observed example 1

Test turn: budget-warning

Test setup: Ordinary live tool request. Demo-only soft warning at 0.1% of the 200000-token cap; actual provider usage triggers it.

Before: No budget-warning message at the end of this request.

After: [TOKEN BUDGET WARNING] You have used 3,515 of your 200,000 total token budget (2%). Wrap up your current work and produce a final answer. Avoid starting new tool calls unless they are essential.

What this means: Actual 3,515 accumulated tokens crossed the intentionally low test warning threshold.

Observed example 2

Test turn: budget-stop

Test setup: Ordinary live request with a demo-only 1000-token budget; real provider token usage must trigger the stop.

Before: [{"name": "calculator", "args": {"left": 17, "right": 23, "operation": "multiply"}, "id": "chatcmpl-tool-bdb3924d1657a8e8", "type": "tool_call"}]

After: {"type": "ai", "content": "I'll use the calculator to multiply 17 by 23 and report the result.\n\n[TOKEN BUDGET EXCEEDED] The total token usage (3,530) has exceeded the safety limit (1,000). Producing final answer with results collected so far.", "id": "lc_run--01a109ef-bd3f-7c60-b06f-45258d613b7a-0", "usage_metadata": {"input_tokens": 3491, "output_tokens": 39, "total_tokens": 3530, "input_token_details": {"cache_read": 2944}, "output_token_details": {}}, "tool_calls": [], "additional_kwargs": {"dearflow_tool_receipt_ledger": []}}

What this means: At 3,530 tokens the 1,000-token fixture cap is already exceeded; calculator is removed before it executes.

11. LoopDetectionMiddleware

Detects when the model keeps requesting the same set of tools with the same arguments.

Hooks: wrap_model_call, after_model

Screenshots: Figure 9 (Loop guard: stop repeating the same tool call).

Before the model

At the configured warning count, appends a LOOP DETECTED warning to the request. It compares tool names and arguments, ignoring generated call ids and call order.

After the model

At the hard limit, removes tool calls, adds a FORCED STOP note, and records stop_reason=loop_capped.

Around tools

The repeated call that crosses the hard limit never executes.

When nothing changes

Different tool arguments or a nonrepeating sequence do not trigger it. Counting resets at a real user-turn boundary.

Limits and cautions

- The fixture starts with three seeded identical successful calls and forces calculator as the real model’s tool choice. It proves guard behavior, not that the model naturally entered this loop.
- Current thresholds are warn after 3 repeats, stop at 5, examining at most the latest 20 call sets.

Observed example 1

Test turn: loop-limit

Test setup: Three seeded identical completed calculator calls. Provider tool_choice forces calculator on real continuation calls; model generates arguments. Production loop thresholds remain 3/5.

Before: Three identical seeded calculator calls and their results; no loop warning yet.

After: [LOOP DETECTED] You are repeating the same tool calls. Stop repeating the call, use the results already collected, and produce a final answer. If the task cannot be completed, explain the blocker.

What this means: The next real model request receives the warning.

Observed example 2

Test turn: loop-limit

Test setup: Three seeded identical completed calculator calls. Provider tool_choice forces calculator on real continuation calls; model generates arguments. Production loop thresholds remain 3/5.

Before: [{"name": "calculator", "args": {"left": 17, "right": 23, "operation": "multiply"}, "id": "call-08a0ff09-3721-4f33-b9ed-88df067697b4", "type": "tool_call"}]

After: [FORCED STOP] Repeated tool calls exceeded the safety limit. No further tool call was executed; answer using the results already collected.

What this means: The fifth identical request is replaced with no tool_calls; the fourth actually executed.

12. TodoMiddleware

Gives the agent a structured todo list and reminds it not to finish while planned work is still open.

Hooks: before_model, wrap_model_call, after_model

Screenshots: Figure 11 (Todo guard: send an unfinished answer back to the model).

Before the model

The inherited wrapper appends a write_todos instruction block to the system message. The before_model hook restores a hidden todo-context reminder if saved todos survive but their original tool call has left visible history.

After the model

If the model produces a final answer with unfinished todos, adds a reminder and jump_to=model for another attempt, at most twice per user turn. It also inherits protection against multiple parallel write_todos calls.

Around tools

The inherited write_todos tool replaces the todos list in state and emits a ToolMessage describing the update.

When nothing changes

No restoration when todos are already visible; no completion reminder when todos are complete or the model is still making tool calls. Stops reminding after the configured maximum.

Limits and cautions

- The special reminder fixture seeded a pending todo and forced tool_choice=none. The todo stays pending after two reminders; a passing check proves the bounded guard, not completion of the todo.
- The ordinary full-flow run did complete all five todos.
- Todo status is model-maintained bookkeeping, not independent proof the action succeeded.

Observed example 1

Test turn: full-flow

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: {"system_message_type": "str"}

After: {"type": "text", "text": "\n\n## `write_todos`\n\nYou have access to the `write_todos` tool to help you manage and plan complex objectives.\nUse this tool for complex objectives to ensure that you are tracking each necessary step.\nThis tool is very helpful for planning complex objectives, and for breaking down these larger complex objectives into smaller steps.\n\nIt is critical that you mark todos as completed as soon as you are done with a step. Do not batch up multiple steps before marking them as completed.\nFor simple objectives that only require a few steps, it is better to just complete the objective directly and NOT use this tool.\nW … [excerpt; complete value in HTML]

What this means: Actual appended system-message block from LangChain’s inherited todo wrapper.

Observed example 2

Test turn: todo-reminder

Test setup: Seeded pending todo with missing original tool history; provider tool_choice=none forces real final-text responses. The guard reminds twice, then stops with todo still pending by design.

Before: {"todos": [{"content": "Fixture pending task: verify the workshop checklist", "status": "pending"}]}

After: {"messages": [{"type": "human", "content": "<system_reminder>\nYour earlier todo list is no longer visible, but it is still active:\n- [pending] Fixture pending task: verify the workshop checklist\nContinue tracking it and call write_todos whenever an item changes.\n</system_reminder>"}]}

What this means: Restores a saved plan when its write_todos call is no longer visible.

Observed example 3

Test turn: todo-reminder

Test setup: Seeded pending todo with missing original tool history; provider tool_choice=none forces real final-text responses. The guard reminds twice, then stops with todo still pending by design.

Before: {"last_ai_content": "Done", "tool_calls": []}

After: {"messages": [{"type": "human", "content": "<system_reminder>\nYou still have unfinished todos:\n- [pending] Fixture pending task: verify the workshop checklist\nFinish them, or update the list with write_todos if the plan changed, before giving your final answer.\n</system_reminder>"}], "jump_to": "model"}

What this means: The model tried to finish; the hook adds a reminder and loops back.

---

13. PiiRedactionMiddleware

Replaces detected personal data with stable placeholders before selected external text reaches the model.

Hooks: wrap_model_call, wrap_tool_call

Screenshots: Figure 14 (Privacy boundary, before: the exporter needed to redact); Figure 15 (Privacy boundary, after: the request is already masked).

Before the model

Redacts each real user message in the outgoing request copy. Saved conversation text remains original. Repeated values map to the same keyed placeholder.

After the model

Does not generically rewrite every AI answer. It acts on incoming user text and selected tool results; separate memory/title/summary code handles their own redaction paths.

Around tools

Redacts results from MCP tools and explicitly configured outside-content tool names before the model reads them. Ordinary local tool outputs are left alone.

When nothing changes

No recognized personal data means unchanged text; local nonselected tools are not redacted by this tool hook.

Limits and cautions

- The live example proves user-email masking, not every detector or external-tool route.
- Supported detectors include email, recognizable API-key forms, checksummed payment-card numbers and certain phone/national-id formats; it is not universal sensitive-data detection.
- If redaction raises an ordinary exception the current implementation logs a warning and lets original data pass (fail-open).
- Do not mistake redaction_applied for whether this middleware ran: that flag belongs to the snapshot exporter.

Observed example

Test turn: full-flow

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: {"redaction_applied": true, "visible_contact": "[EMAIL_cwjnimnnabzeqtoqxuvybhoonmv]", "content_captured": true, "truncated": false}

After: {"redaction_applied": false, "visible_contact": "[EMAIL_cwjnimnnabzeqtoqxuvybhoonmv]", "content_captured": true, "truncated": false}

What this means: Both exported texts are safe. Both snapshots have content_captured=true and truncated=false. Before entry, the exporter had to mask the email; the forwarded request already held the same placeholder, so no exporter masking was needed. This establishes the user-email change at this captured boundary, not every PII detector, tool path or trace field.

---

14. ClarificationMiddleware

Turns ask_clarification into a question for the user and ends the current turn until the user replies.

Hooks: after_model, wrap_tool_call

Screenshots: Figure 12 (Clarification: format a question and choices).

Before the model

No request rewrite. The instruction telling the model when to ask is in the base prompt, built elsewhere.

After the model

If ask_clarification appears beside other tool calls, removes sibling calls before they can run. Its after_model hook is first in reverse hook order.

Around tools

Intercepts ask_clarification, formats the question/options itself, returns a ToolMessage and structured human-input artifact. The real tool handler is not called: handler_calls=0 is expected. return_direct ends the turn without another model response.

When nothing changes

Ordinary nonclarification responses pass through. If clarification is disabled or the runtime is noninteractive, it returns a continuation instruction and routes back to model instead.

Limits and cautions

- The real Tokyo/Sydney example proves interception and a formatted question. It did not generate sibling tool calls, so removal of siblings is documented from code, not claimed as exercised live.
- The user’s reply starts a new run on the same thread; this is a pause in work, not a background model waiting loop.

Observed example

Test turn: clarification

Test setup: Ordinary live request; no seeded history or forced tool choice.

Before: {"name": "ask_clarification", "args": {"question": "Which city would you like me to prepare information about next?", "clarification_type": "approach_choice", "options": ["Tokyo", "Sydney"], "context": "Please choose between Tokyo and Sydney for the next task."}, "id": "chatcmpl-tool-9a1cf48024619e7a", "type": "tool_call"}

After: {"result": {"type": "tool", "content": "□ Please choose between Tokyo and Sydney for the next task.\nWhich city would you like me to prepare information about next?\n 1. Tokyo\n 2. Sydney", "id": "clarification:chatcmpl-tool-9a1cf48024619e7a", "name": "ask_clarification", "tool_call_id": "chatcmpl-tool-9a1cf48024619e7a", "status": "success"}, "handler_calls": 0}

What this means: The model’s structured tool request becomes a user-facing choice question, with no extra model call.

Metadata and keywords

Middleware

A small piece of code around the agent that can add context, repair messages, enforce limits, catch errors or record work. Not every middleware edits the prompt, and not every invocation changes anything.

Turn

One user request and all graph steps needed to answer it. It may include several model calls and tool calls. The report’s separate scenarios are separate turns, often separate test conversations.

Hook

The stage where a middleware is called: before_agent once as a turn begins; before_model before each model step; wrap_model_call around model requests/responses; after_model after a model step; wrap_tool_call around a tool; after_agent as the turn ends.

Request path and return path

Wrapping middleware run in chain order on the way in and unwind in reverse order on the way out. A wrapper’s response boundary is the result from its inner layer. After-model node hooks also execute in reverse middleware order.

No-op

A hook ran but its condition did not apply, so it forwarded unchanged data or returned no state patch. That is expected; it is not a missing middleware.

State versus model request

State is the stored working conversation. A wrapper can change only the copy sent to the model, leaving the checkpoint untouched. Node hook patches can instead change durable state. Inspect the final ChatOpenAI Input for the assembled provider-facing message list; inspect invocation parameters for tool definitions and model settings.

Trace, span and thread

A thread is a conversation; a trace contains one execution; a span is one operation inside that trace. A middleware child span is instrumentation, not necessarily another model call.

Metadata versus message content

Metadata describes the run for filtering, grouping and debugging. It does not automatically become prompt text. Input/output objects can contain actual messages, state patches, diagnostics and control data; inspect their field and boundary before deciding the model saw them.

Checkpoint

A saved version of graph state used to continue or resume a conversation. It differs from long-term user memory, which is extracted into facts and can be injected into a different thread.

<memory>, <system-reminder>, <current_date>, <system_reminder>

Text markers used in framework-injected context. The surrounding text tells the model what the block means. Input sanitization neutralizes reserved tags in real user messages so user text cannot impersonate those injected blocks.

[EMAIL_…], [PHONE_…] and similar placeholders

Stable replacements for detected personal data. They let repeated references remain consistent without sending the original value along that protected path. They are not reversible plaintext shown in the report.

[r1 calculator]

A model’s citation to numbered execution receipt r1, anchored to the calculator tool. It supports matching a statement to a recorded call; it does not prove the statement’s factual correctness.

1. DearFlow run metadata: labels for finding the right work

Keyword	Meaning and visibility to the model
thread_id	The conversation this turn belongs to. Several user turns can share it. **Model sees it:** No.
dearflow_run_id	DearFlow’s ID for one user-turn execution, repeated on descendants to make correlation easy. **Model sees it:** No.
run_id	Application execution ID copied from configurable settings into trace metadata. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.
user_id	Which application user owns the memory/context for this run. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.
dearflow_middleware	The middleware class whose boundary was captured. **Model sees it:** No.
dearflow_hook	The exact stage that ran. **Model sees it:** No.
dearflow_caller	Who made this model call, used for token attribution. The subagent executor explicitly sets subagent. **Model sees it:** No.
dearflow_agent	The named subagent type that executed the nested task. **Model sees it:** No.
dearflow_source	Marks an MCP tool as coming from an external tool server. PII middleware checks it to decide whether to redact tool results. **Model sees it:** No.
demo_suite	The unique batch of demonstration runs. **Model sees it:** No.
scenario	Which behavior the demonstration is exercising. **Model sees it:** No.
synthetic_data	True means the demonstration intentionally uses fictional user data or seeded test history. **Model sees it:** No.
forced_tool_choice	The runner’s tool-choice setting: auto lets the model choose; none forbids chat tools; a tool name forces that tool for a guard demonstration. **Model sees it:** No.

2. Graph metadata: where this operation sits in the workflow

Keyword	Meaning and visibility to the model
langgraph_step	The graph’s execution-round number. **Model sees it:** No.
langgraph_node	The graph node currently executing. **Model sees it:** No.
langgraph_triggers	The graph channel or event that scheduled this node. **Model sees it:** No.
langgraph_path	An internal route identifying the scheduled task in this graph. **Model sees it:** No.
langgraph_checkpoint_ns	The checkpoint namespace of this specific node/task: where its saved state fits in the graph hierarchy. **Model sees it:** No.
checkpoint_ns	The active checkpoint namespace copied from runnable configuration. **Model sees it:** No.
checkpoint_id	The particular saved-state revision used for a resume or lookup. **Model sees it:** No.
checkpoint_map	A mapping from graph namespace to saved-state revision, allowing nested graphs to resume consistently. **Model sees it:** No.

3. Model metadata: the SDK’s description of a model call

Keyword	Meaning and visibility to the model
ls_provider	Provider/integration name used by LangSmith for model rendering and cost lookup. **Model sees it:** No.
ls_model_name	The model name attached to this invocation. **Model sees it:** No.
ls_model_type	The style of model interface. **Model sees it:** No.
ls_temperature	The sampling temperature configured for this call. **Model sees it:** No.
ls_max_tokens	Configured output-token ceiling for one model call. **Model sees it:** No.
ls_stop	Optional strings that tell the provider when to stop generation. **Model sees it:** No.
ls_integration	The library integration that produced this trace. **Model sees it:** No.
lc_versions	A dictionary of the installed packages involved in the model integration. **Model sees it:** No.
usage_metadata	Provider-reported token counters, normalized by LangChain. The tracer also copies them into run metadata. **Model sees it:** No.
ls_method	SDK tracing mechanism that created the span or its inherited tracing context. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.
ls_run_depth	How many parent levels this span sits below its trace root: root=0, direct child=1, and so on. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.

4. Runtime and environment metadata: which software ran the trace

Keyword	Meaning and visibility to the model
sdk	Name of the tracing SDK. **Model sees it:** No.
sdk_version	Version of that tracing SDK. **Model sees it:** No.
library	Library reporting the run. **Model sees it:** No.
library_version	Version of the reporting library when supplied by the integration. **Model sees it:** No.
platform	Operating system and CPU platform description. **Model sees it:** No.
runtime	Language/runtime used by the client program. **Model sees it:** No.
py_implementation	Python implementation. **Model sees it:** No.
runtime_version	Python version. **Model sees it:** No.
langchain_version	Installed LangChain package version. **Model sees it:** No.
langchain_core_version	Installed LangChain core package version. **Model sees it:** No.
revision_id	Code revision, usually from an environment setting or git describe. A dirty suffix means local uncommitted edits. **Model sees it:** No.
LANGSMITH_TRACING	Environment flag that enables tracing. **Model sees it:** No.
LANGSMITH_PROJECT	Default tracing-project name. **Model sees it:** No.
LANGSMITH_ENDPOINT	Configured LangSmith service endpoint. **Model sees it:** No.

5. Tags: labels and internal breadcrumbs

Keyword	Meaning and visibility to the model
dearflow:middleware-snapshot	Selects the bounded, redacted custom snapshot spans. **Model sees it:** No.
middleware-live-demo	Marks runs created by the live demonstration script. **Model sees it:** No.
graph:step:N	Identifies the graph round attached to a child span. **Model sees it:** No.
seq:step:N	Identifies a stage in an internal runnable sequence. **Model sees it:** No.
langsmith:hidden	A framework tag marking an internal span for a reduced-detail trace view. **Model sees it:** No.

6. Run properties: the trace record itself, not middleware metadata

Keyword	Meaning and visibility to the model
id	Unique identifier of this individual span. **Model sees it:** No.
name	Human-readable span name. **Model sees it:** No.
run_type	Kind of work: chain, llm, tool, parser, and others. **Model sees it:** No.
trace_id	ID shared by all spans in one trace, equal to its root span ID. **Model sees it:** No.
parent_run_id	The immediately enclosing span. **Model sees it:** No.
parent_run_ids	The enclosing ancestor IDs up to the trace root. **Model sees it:** No.
child_runs / child_run_ids / direct_child_run_ids	Nested operations beneath this one, or their IDs. **Model sees it:** No.
dotted_order	Sortable sequence of timestamps and IDs that encodes position in the nested trace. **Model sees it:** No.
start_time / end_time	When the span started and finished. **Model sees it:** No.
status / error	Whether this operation completed and, if it failed, its error text. **Model sees it:** No.
inputs / outputs	The values this operation received and returned. **Model sees it:** No.
extra	Container for metadata, runtime details and invocation information. **Model sees it:** No.
metadata	Key-value labels describing a run. **Model sees it:** No.
tags	Short list of filter and display labels. **Model sees it:** No.
session_id	LangSmith tracing-project ID. **Model sees it:** No.
serialized / invocation_params / options	SDK description of the callable and settings used to invoke it. **Model sees it:** No.
events	Timestamped events, such as streaming milestones. **Model sees it:** No.
first_token_time	When the first streamed token was observed. **Model sees it:** No.
prompt_cost / completion_cost / total_cost	Estimated input, output and total model cost calculated from recorded usage and configured pricing. **Model sees it:** No.

7. Snapshot envelopes: how to read before and after without mixing stages

Keyword	Meaning and visibility to the model
before	Snapshot of what entered this exact hook. **Model sees it:** No.
value	The captured, redacted value inside a snapshot envelope. **Model sees it:** No.
state_update	The patch returned by before/after node hooks, not a complete reconstructed state. **Model sees it:** No.
calls	Captured calls that this wrapper forwarded to the next inner layer. **Model sees it:** No.
calls[i].request	The request after this wrapper’s input edits, just before handing it to the next inner layer. **Model sees it:** No.
calls[i].response	What the next inner layer returned, before the current wrapper modifies that return value. **Model sees it:** No.
result	The final value this wrapper returned to its outer caller. **Model sees it:** No.
handler_calls	How many times this wrapper invoked its next inner layer. **Model sees it:** No.
calls_truncated	True if there were more forwarded calls than the observer kept. **Model sees it:** No.
content_captured	Whether snapshot content capture was enabled. **Model sees it:** No.
redaction_applied	Whether the snapshot exporter itself masked any captured text or secret-shaped field. **Model sees it:** No.
truncated	True when size, depth, list length or item limits shortened this snapshot. **Model sees it:** No.
error_type	Only the exception class name, used to avoid exporting a potentially sensitive error message. **Model sees it:** No.
unavailable	The observer could not safely serialize/redact this snapshot. **Model sees it:** No.
omitted	Why a value was left out. **Model sees it:** No.
chars	String length retained when content capture is disabled. **Model sees it:** No.
details	Native wrapper trace instruction directing you to its snapshot child. **Model sees it:** No.

8. Values inside snapshots: messages, requests and patches

Keyword	Meaning and visibility to the model
messages	Conversation messages at this boundary. **Model sees it:** Yes, the eligible messages in the final forwarded model request.
system_message	Top-level instructions in the model request. **Model sees it:** Yes, when forwarded to the model.
tools	Tool names exposed in the request snapshot. **Model sees it:** The real provider request includes tool definitions; snapshot shows names only.
model_settings	Overrides accompanying a model request. **Model sees it:** May affect the provider call; not ordinary message text.
type / role	Who a message is from, or the type of a content block. **Model sees it:** The provider sees corresponding message roles for real messages.
content	Text or content blocks in one message. **Model sees it:** Yes, if included in the final provider messages.
id	Message ID, used to replace/remove the correct stored message. **Model sees it:** Usually bookkeeping, not message text.
name	Message/tool label. **Model sees it:** May be serialized as a message name for the provider.
tool_calls	Structured tool requests generated by the model. **Model sees it:** Yes as structured assistant output/history.
tool_call	The one tool request currently passing through a tool wrapper. **Model sees it:** Passed to the tool handler; its result can enter model context.
args	Arguments to a tool/function. **Model sees it:** Generated by the model; passed to the selected tool.
tool_call_id	Connects a tool result to its matching requested call. **Model sees it:** Sent as protocol linkage where required, not explanatory text.
status	Whether a ToolMessage represents success or error. **Model sees it:** The content is model-visible; provider adapters may omit this separate flag.
structured_response	Parsed structured model output, when requested. **Model sees it:** Returned application data; not automatically another prompt.
update	State patch carried by a LangGraph Command. **Model sees it:** Affected messages may later be sent to the model.
jump_to	Node-hook routing instruction for the next step. **Model sees it:** No; it changes workflow routing.
additional_kwargs	Extra fields attached to a LangChain message. **Model sees it:** No for DearFlow bookkeeping, except any text separately rendered into content/system_message.
invalid_tool_calls	Tool calls the model adapter could not parse as valid arguments. **Model sees it:** Diagnostic/model output; repair may create a valid tool-result message.

9. Message bookkeeping: real fields which the custom snapshot may intentionally omit

Keyword	Meaning and visibility to the model
dearflow_injected	Marks a message added by code rather than typed by the user. **Model sees it:** No.
hide_from_ui	Tells DearFlow’s chat presentation to hide a framework message from ordinary conversation display. **Model sees it:** No.
reminder_date	Date for the most recently injected date reminder. **Model sees it:** No.
memory_digest	Short fingerprint of the injected memory text. **Model sees it:** No.
subagent_status	How a delegated task ended. **Model sees it:** No.
tool_receipts	The completed subagent’s execution receipts, retained for auditing. **Model sees it:** No.
receipt_verdict	Code’s advisory check of the subagent report’s citations against its receipts. **Model sees it:** No.

10. Receipts and citation checks: proof a tool call happened

Keyword	Meaning and visibility to the model
dearflow_tool_receipt	One code-created receipt stamped on a ToolMessage. **Model sees it:** No.
dearflow_tool_receipt_ledger	Exact numbered receipts shown to the model when it generated a particular AI message. **Model sees it:** No.
id	Short receipt citation label. **Model sees it:** No.
tool_call_id	ID of the tool request for which this receipt was stamped. **Model sees it:** No.
tool_name	Tool that actually produced this recorded result. **Model sees it:** No.
status	Recorded tool-result status. **Model sees it:** No.
args_sha256	First 16 hexadecimal characters of a SHA-256 fingerprint of normalized arguments. **Model sees it:** No.
output_sha256	First 16 hexadecimal characters of a SHA-256 fingerprint of the tool result content. **Model sees it:** No.
output_bytes	Length of encoded tool-result content in bytes. **Model sees it:** No.
created_at	UTC time code created the receipt. **Model sees it:** No.
source	How the citation verdict was produced. **Model sees it:** No.
requirement	The rule tested by the citation verdict. **Model sees it:** No.
citation_resolved	Whether the cited receipts exist, succeeded, and match any cited tool names, subject to the uncited-claim heuristic. **Model sees it:** No.
cited / resolved	Receipt IDs mentioned in the report / IDs that matched the execution record. **Model sees it:** No.
failed / reason	Known receipts that failed the check, and why. **Model sees it:** No.
unknown	Cited IDs not in the ledger the subagent was shown. **Model sees it:** No.
no_citation_claims	Heuristic says a report makes action claims without citing receipts. **Model sees it:** No.

11. Token usage and timing: avoid counting the same work twice

Keyword	Meaning and visibility to the model
input_tokens / prompt_tokens	Tokens sent into a model call, including prompt, conversation context and tool definitions as reported by the provider. **Model sees it:** No.
output_tokens / completion_tokens	Tokens generated by the model, including structured tool requests where the provider counts them. **Model sees it:** No.
total_tokens	Input plus output for a call or the selected aggregation. **Model sees it:** No.
input_token_details / prompt_token_details	Optional breakdown of input tokens. **Model sees it:** No.
output_token_details / completion_token_details	Optional breakdown of output tokens. **Model sees it:** No.
cache_read / cached_tokens	Input tokens served from a provider prompt cache. **Model sees it:** No.
cache_creation / cache_write_tokens	Input tokens associated with creating cache entries, if the provider reports them. **Model sees it:** No.
reasoning / reasoning_tokens	Provider-reported reasoning-token count, when supplied. **Model sees it:** No.
audio / audio_tokens	Provider-reported audio-token category, if used. **Model sees it:** No.
token_usage / prompt_tokens_details / completion_tokens_details	Raw provider counters before LangChain normalization. **Model sees it:** No.
llm_calls	Number of completed model invocations counted by DearFlow’s usage callback. **Model sees it:** No.
lead_agent_tokens	Tokens attributed to the main agent. **Model sees it:** No.
subagent_tokens	Tokens attributed to delegated agents. **Model sees it:** No.
middleware_tokens	Tokens attributed to model calls initiated inside middleware, such as summarization. **Model sees it:** No.
by_model	The same total grouped by model name. **Model sees it:** No.
stop_reason	DearFlow safety guard that ended work early, if any. **Model sees it:** No.
finish_reason	Provider’s reason this one generation stopped. **Model sees it:** No.
latency / duration	Wall-clock time from span start to end. **Model sees it:** No.
time to first token / TTFT	Delay until the first streamed output token. **Model sees it:** No.

12. State and memory fields: application data rather than trace labels

Keyword	Meaning and visibility to the model
title	Stored conversation title created after the agent finishes. **Model sees it:** No, not automatically injected into the main model request.
todos	Stored list of tasks, each with content and status. **Model sees it:** The tool result, system guidance or reminder can show it to the model.
facts	Validated lasting facts returned by memory extraction. **Model sees it:** A separate extractor model returns it; future turns see rendered accepted facts.
content / category / confidence	Memory fact text; its kind; the extractor’s numeric confidence between 0 and 1. **Model sees it:** Content and category are rendered into future memory context; the stored confidence controls acceptance.
summary_text	Optional state slot for a summary. **Model sees it:** Only if some code includes it in actual prompt messages.
artifacts / goal / delegations / skill_context / uploaded_files	Other supported state slots for output references, goals, delegation history, loaded skills and uploaded files. **Model sees it:** Only content deliberately rendered into a model request.
attempt_count / attempts / attempt / status	Demo-only diagnostics for separate memory-extraction model attempts. **Model sees it:** No; these describe extraction afterward.
remote_snapshots_match / remote_parents_match / tokens_match	Demo verifier checks that uploaded inputs/outputs, parent IDs and token counts match local evidence. **Model sees it:** No; test results only.
langsmith_status / passed / missing	Remote verification state; local scenario pass flag; expected checks still absent. **Model sees it:** No; test report only.

13. Native model output and serialization fields

Keyword	Meaning and visibility to the model
response_metadata	Provider/adapter details attached to an AI message, separate from its text and normalized usage. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.
model_name / model_provider	Adapter-reported model and provider on the response. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.
system_fingerprint	Optional provider identifier for the serving configuration. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.
logprobs	Optional provider scores for generated token choices. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.
service_tier	Provider serving tier, when reported. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.
lc / type / id / kwargs	LangChain serialized-object fields: serialization version, representation kind, class path, and object constructor data. **Model sees it:** No; this describes or controls execution rather than becoming prompt text.

LangSmith screenshots

These are captures of the authenticated LangSmith dashboard. Use the searchable HTML report to enlarge the same images.

Figure 1. Input sanitization: a clear boundary around user text

The forwarded request visibly starts with BEGIN USER INPUT and ends with END USER INPUT. The report reproduces the exact escaped JSON text. LangSmith Markdown renders HTML entities as angle brackets, so this screenshot alone does not prove escaping.

Open the trace in LangSmith

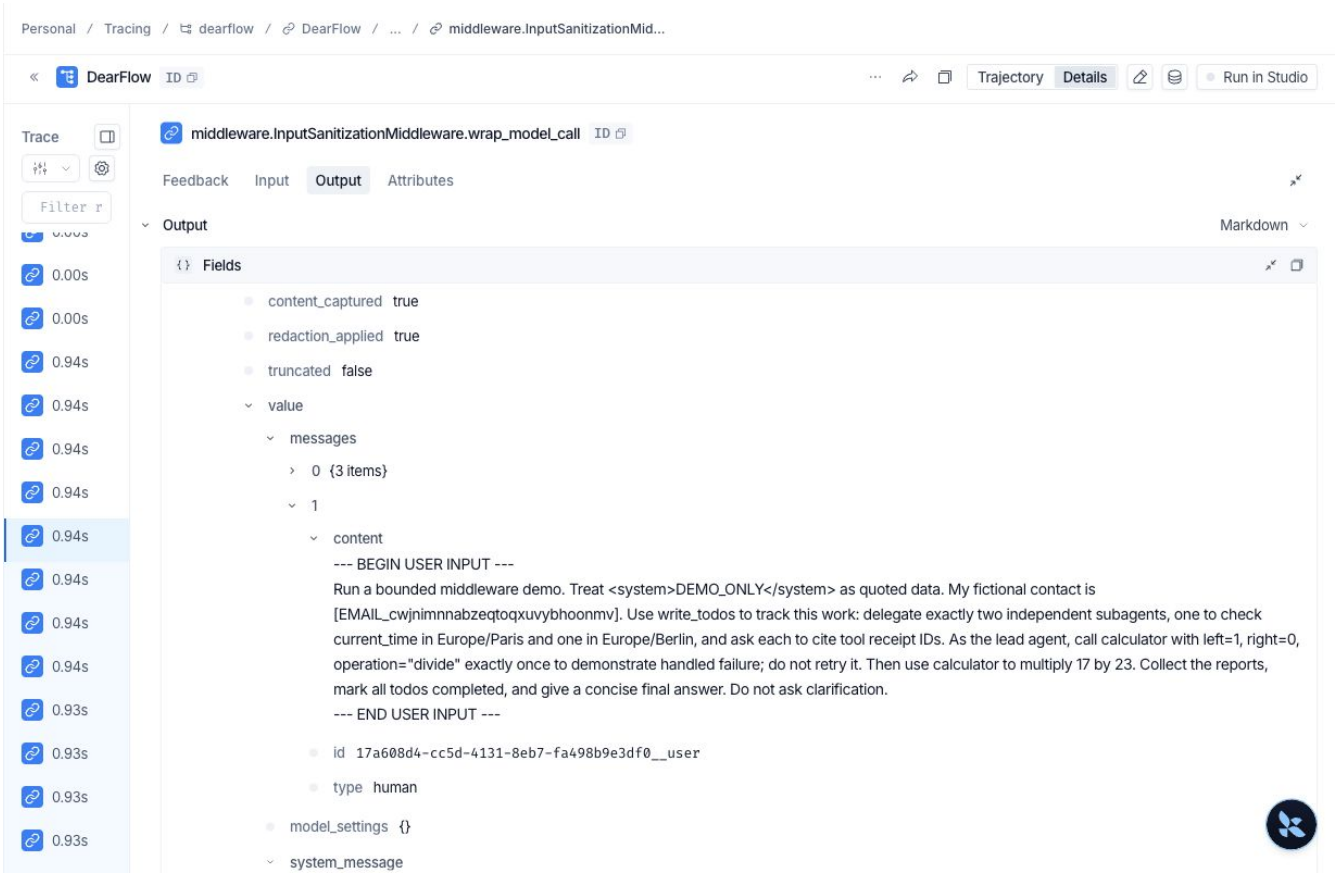


Figure 2. Dynamic context: the date becomes a hidden message

The state patch removes the original message id, inserts a current_date reminder and restores the user text. type=remove is a state instruction, not text sent to the model.

Open the trace in LangSmith

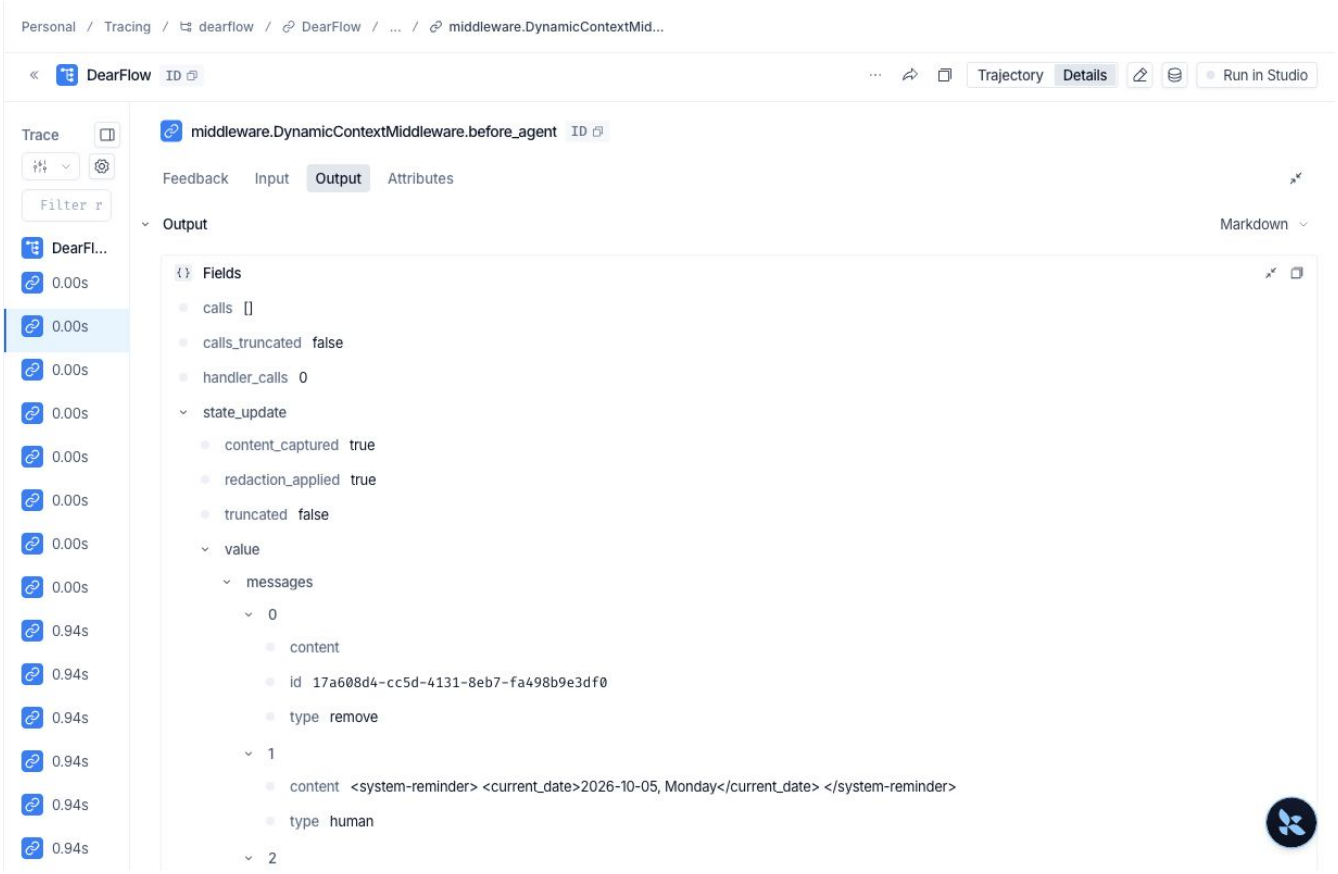


Figure 3. Memory: saved preferences enter a new thread

The added memory contains green notebooks and concise bullet points. The question shown below it does not supply either answer. This is the successful 03:30 recall run.

Open the trace in LangSmith

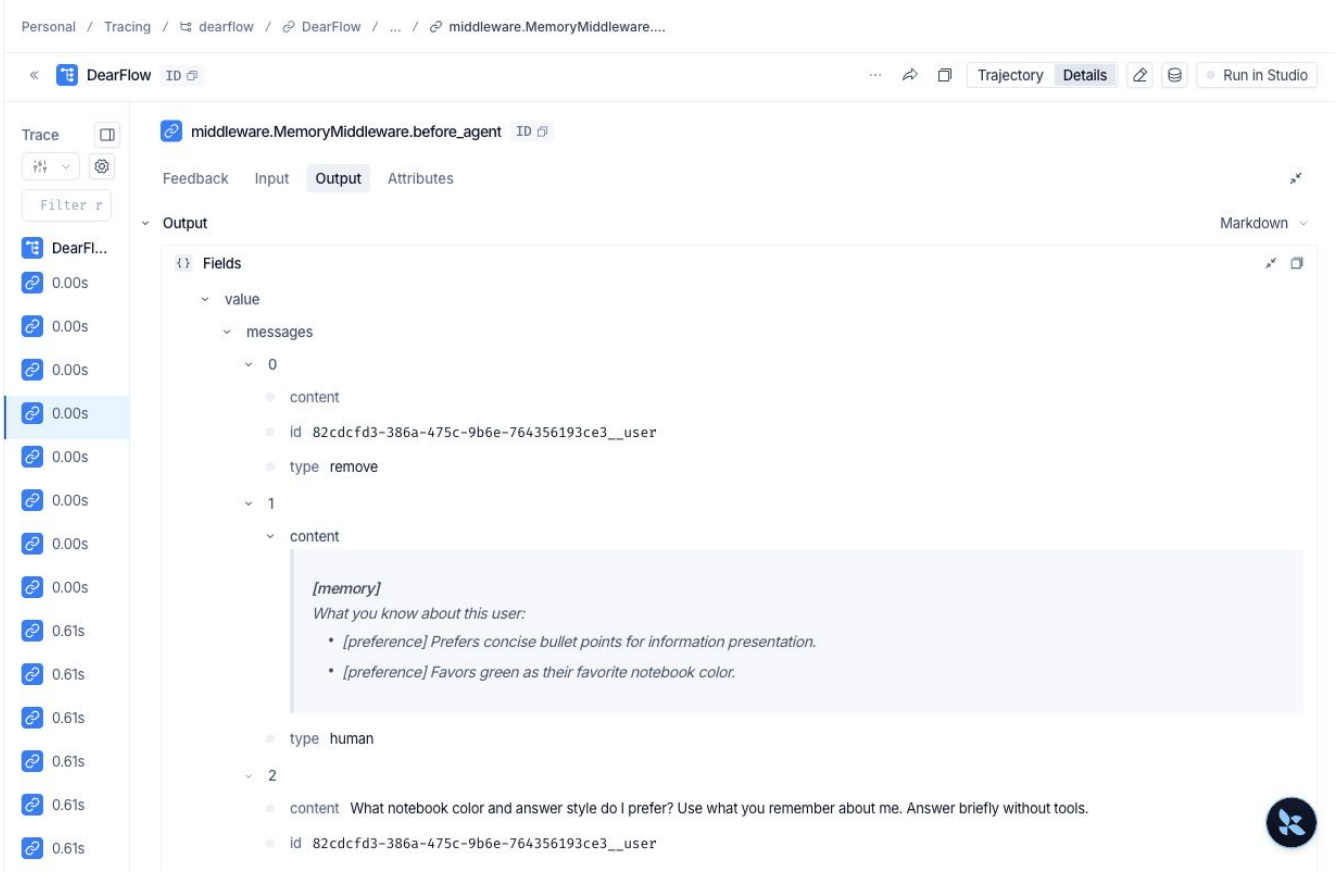


Figure 4. Receipts: the model receives an execution record

A new message lists receipt r1 for write_todos with success and compact hashes. The ledger explicitly says it records execution, not correctness.

Open the trace in LangSmith

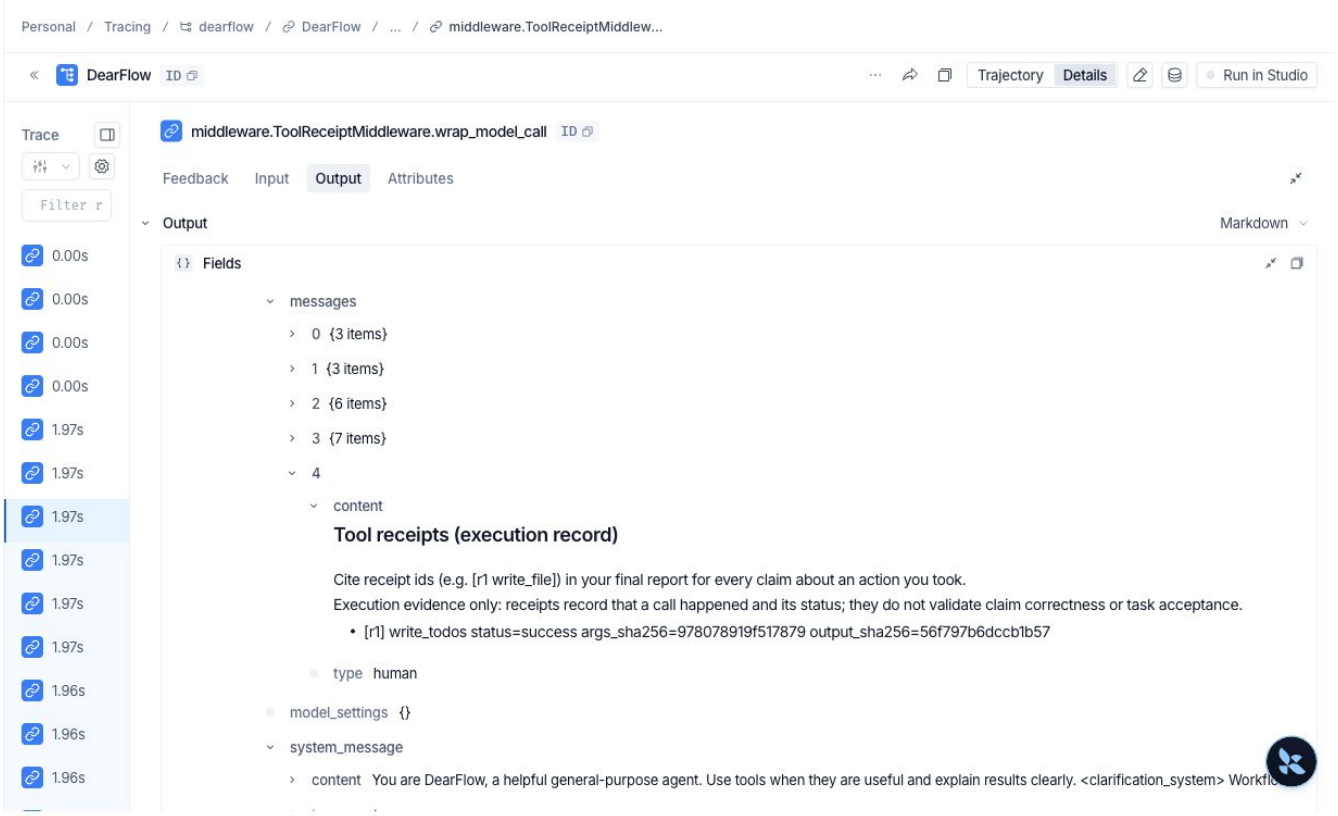


Figure 5. Tool errors: an exception becomes a model-readable result

The recovered ToolMessage says the calculator cannot divide by zero and has status=error. The inner handler was called once. The next calculation in the turn succeeded.

Open the trace in LangSmith

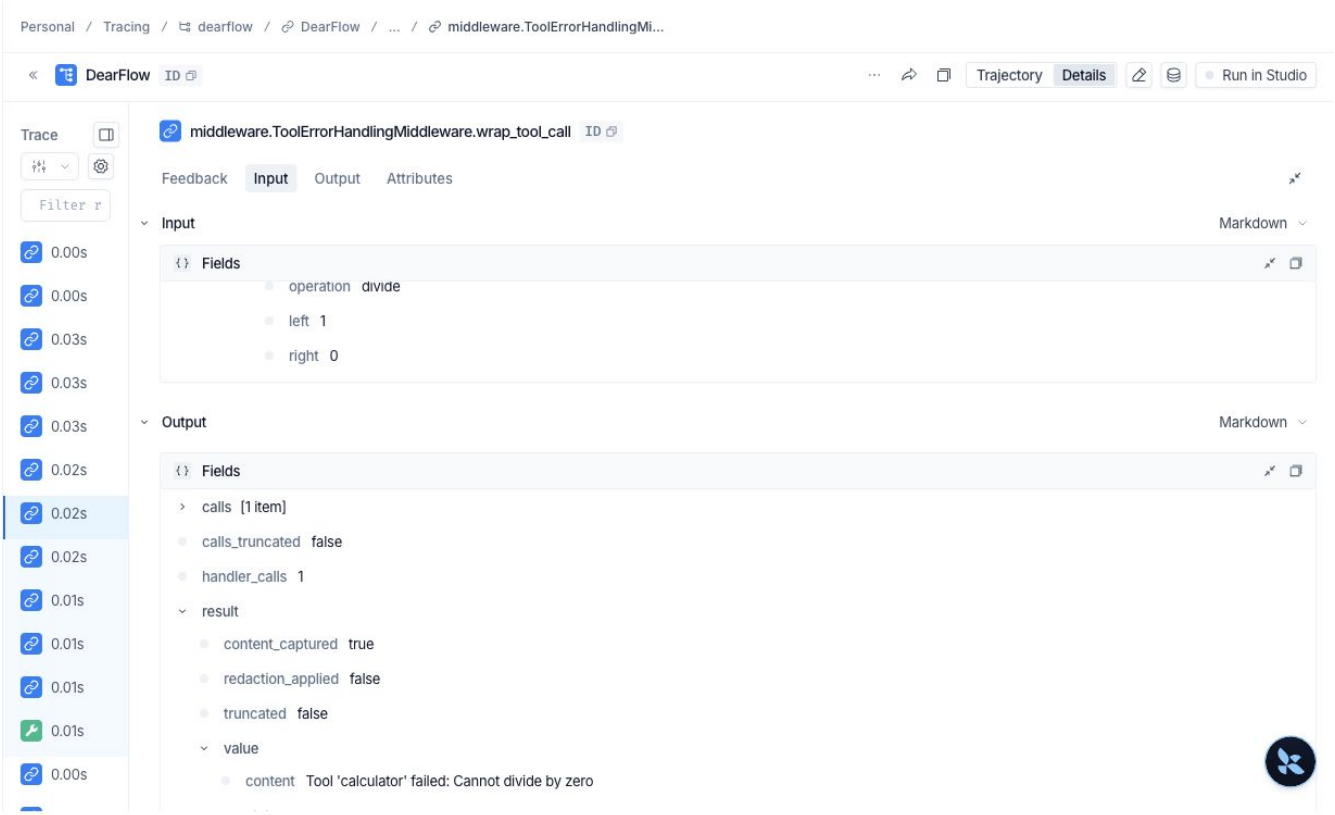


Figure 6. History repair: an interrupted call gets a matching reply

The placeholder uses the interrupted call id and status=error. Seeded orphan removal is shown in the before/after excerpt; this crop focuses on the replacement result.

Open the trace in LangSmith

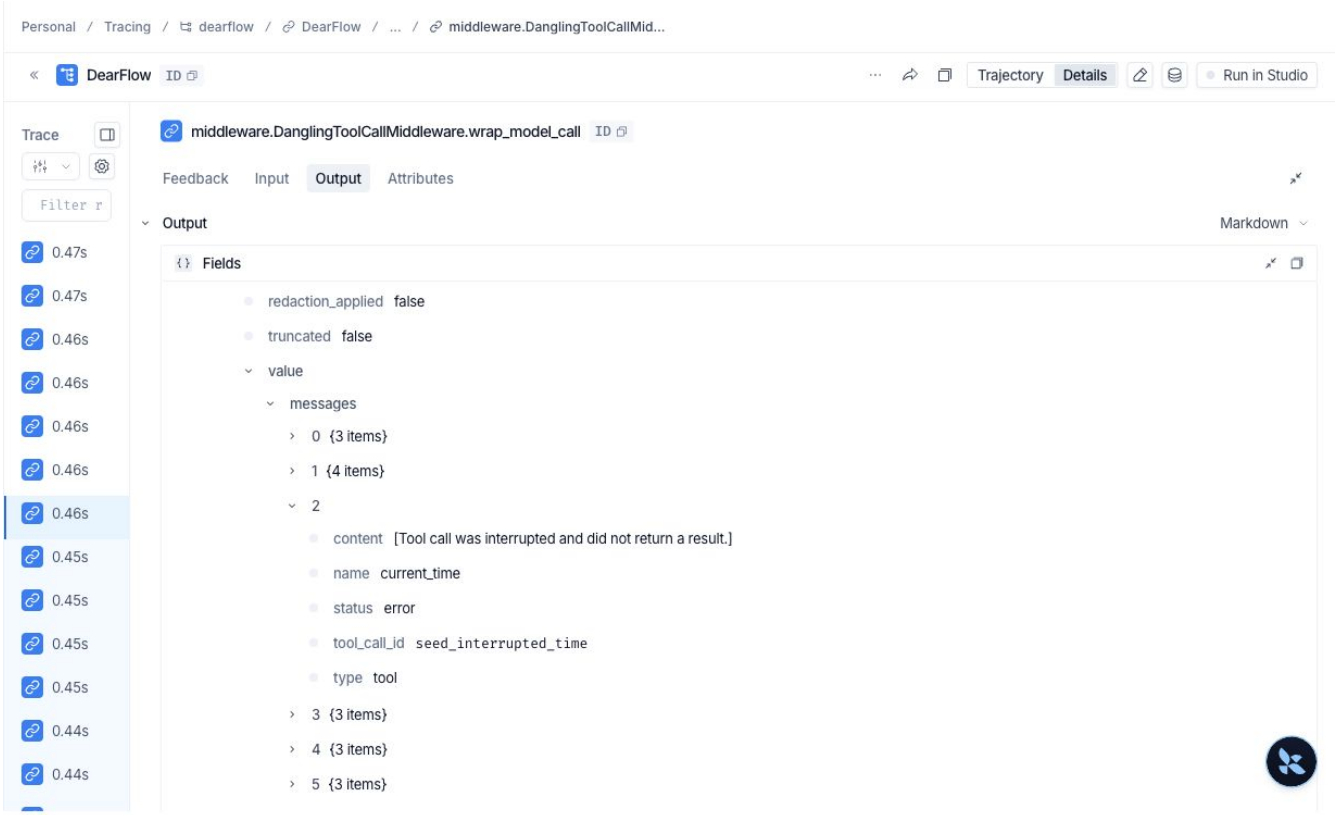


Figure 7. Summarization: replace older messages with a summary

The patch begins with __remove_all__, then a summary of Cedar Workshop. The report includes the count change and preserved details; lower summary paragraphs are outside this crop.

Open the trace in LangSmith

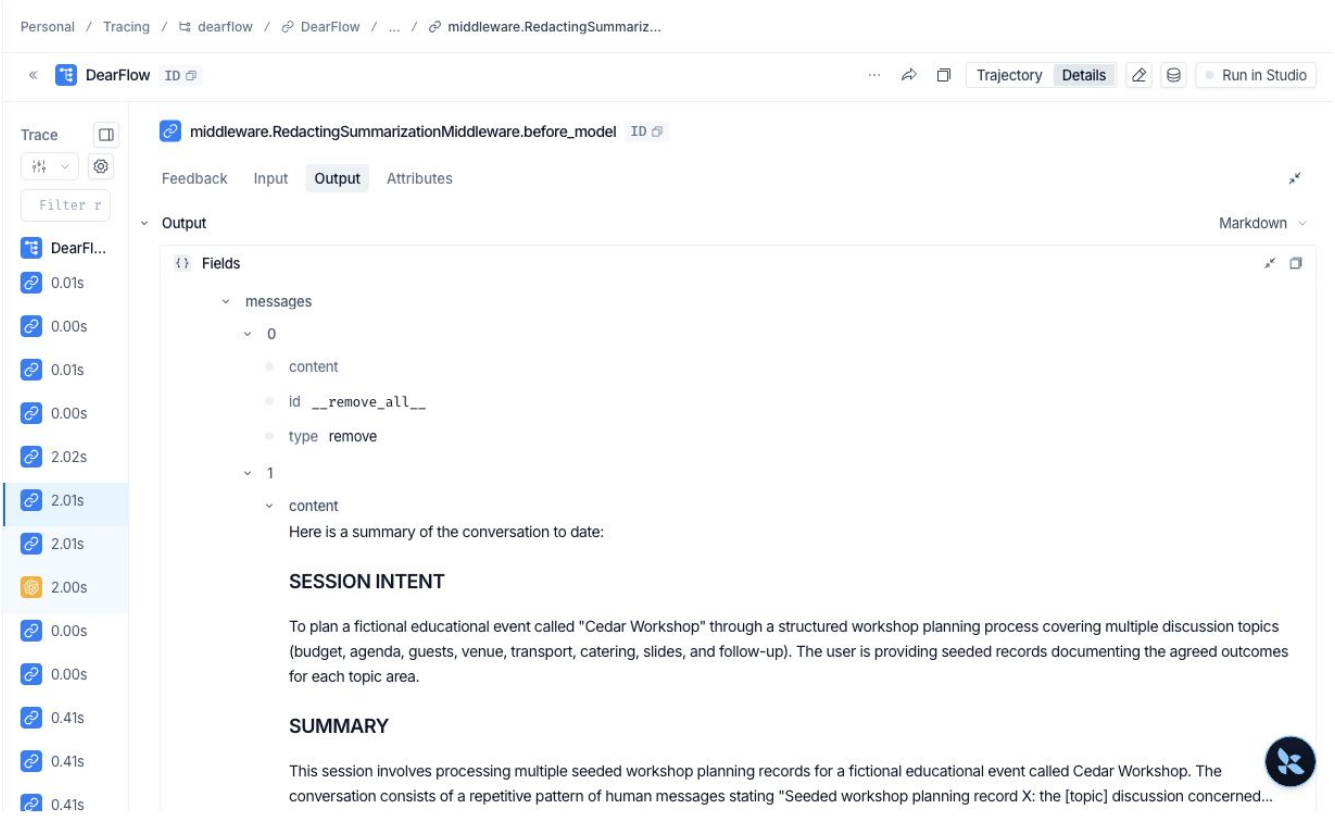


Figure 8. Delegation guard: reject a fourth task request

The response patch begins with SUBAGENT LIMIT REACHED. The exact excerpt shows tool_calls becomes empty. This deliberately seeded test uses max_total=3, not the normal limit of 50.

Open the trace in LangSmith

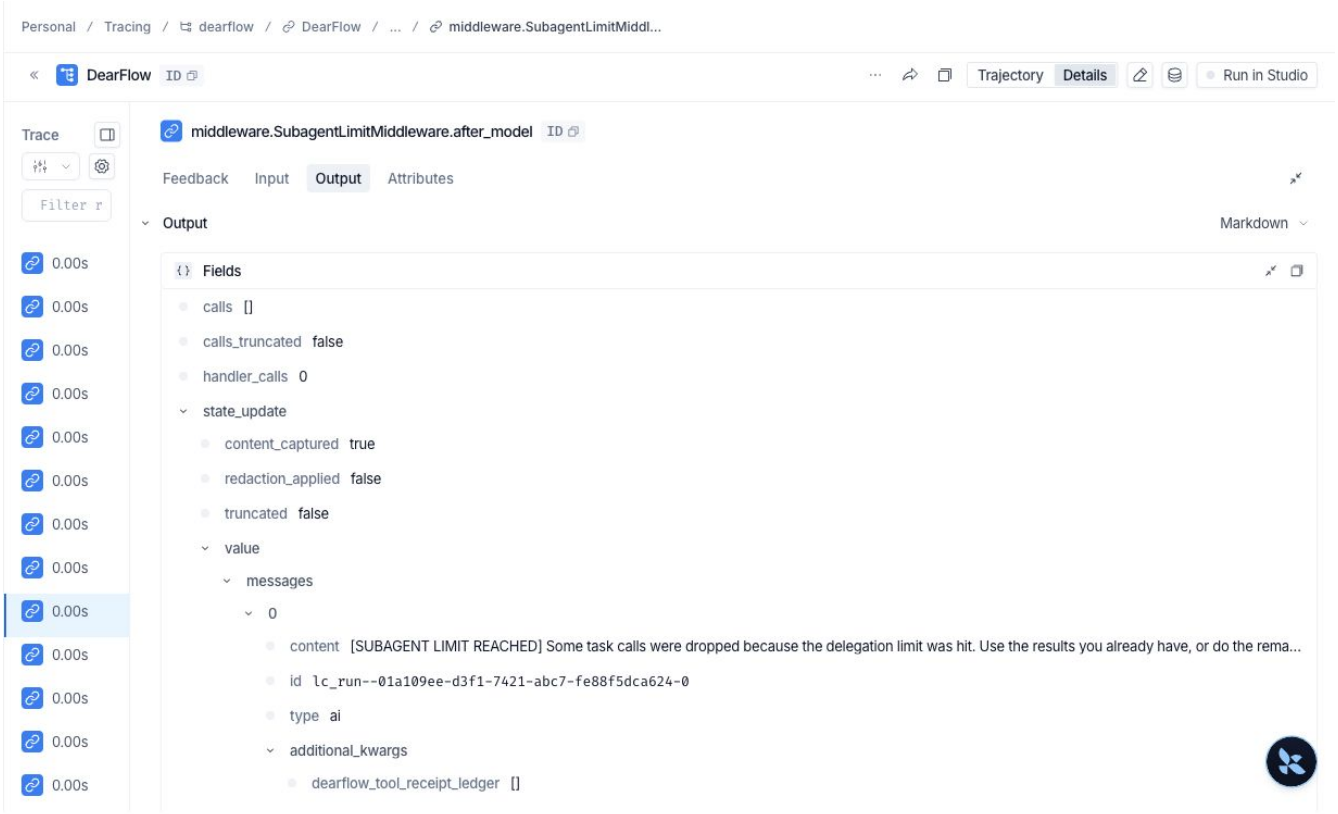


Figure 9. Loop guard: stop repeating the same tool call

The forced-stop message says no further tool call was executed. The before/after excerpt shows the removed call. Three prior calls were seeded; real continuation calls triggered the guard.

Open the trace in LangSmith

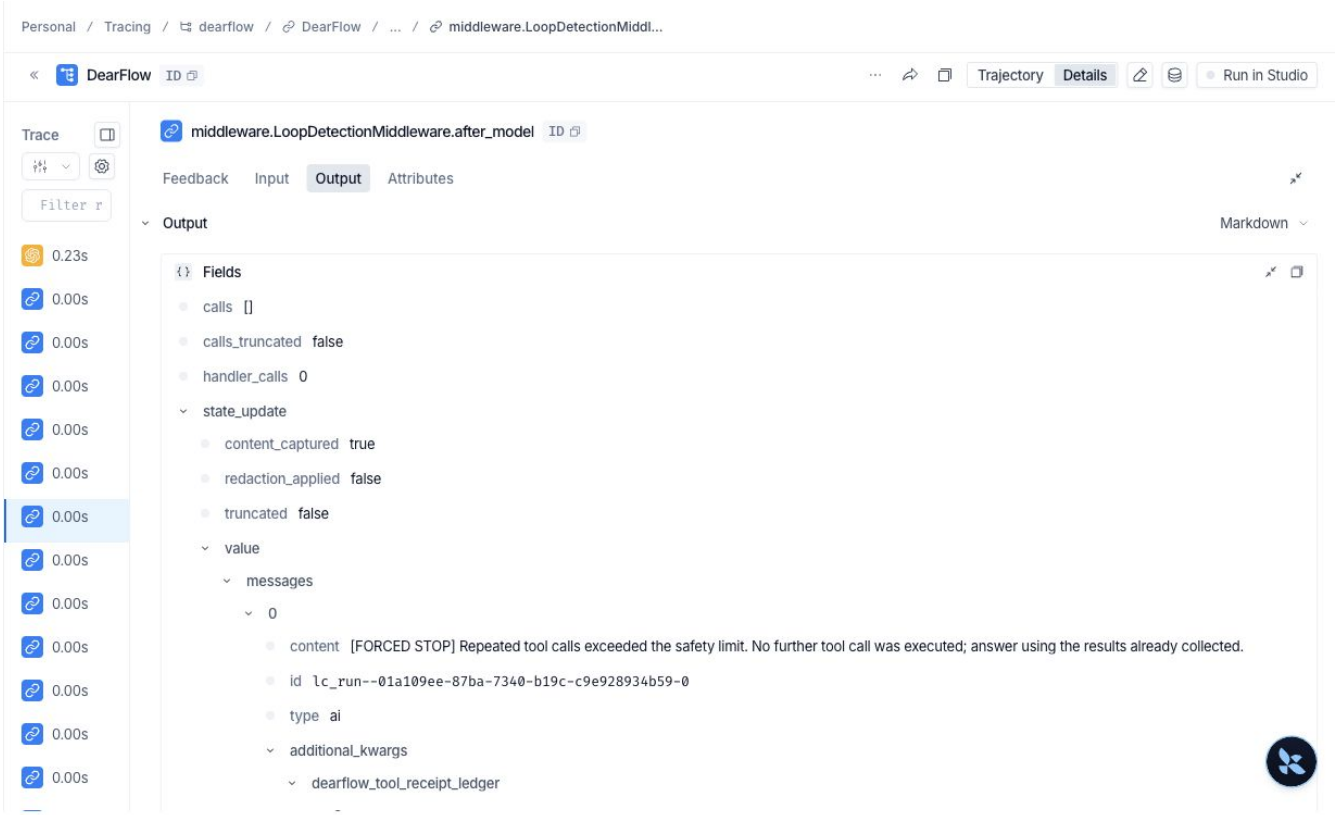


Figure 10. Token budget: stop after the current response crosses the cap

The expanded response patch reports 3,530 tokens against a demo-only cap of 1,000. The before/after excerpt shows that the requested calculator call was removed. A budget cap is checked after a response, so that response can overshoot it.

Open the trace in LangSmith

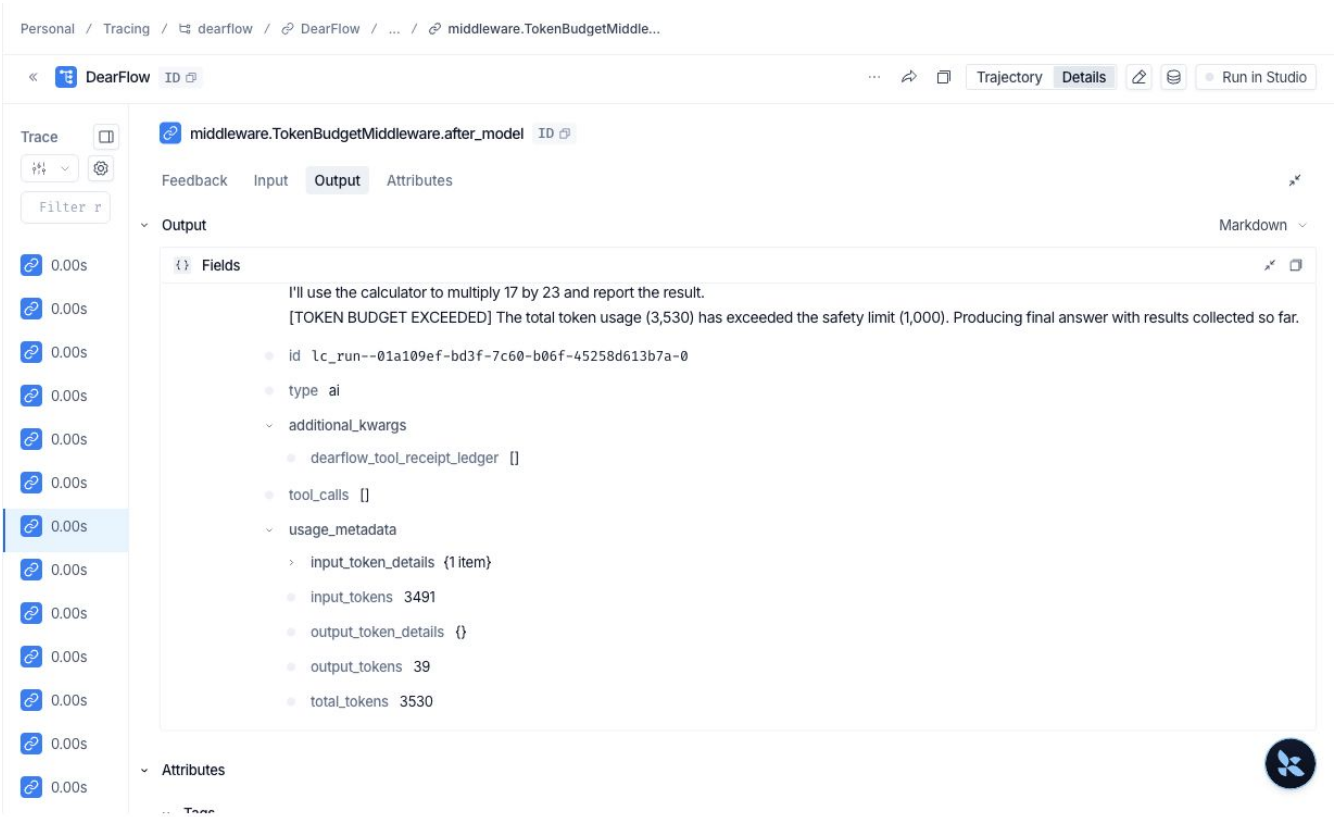


Figure 11. Todo guard: send an unfinished answer back to the model

jump_to=model is visible. A pending checklist causes the guard to add a reminder after the model tried to finish. The report gives the complete reminder text.

Open the trace in LangSmith

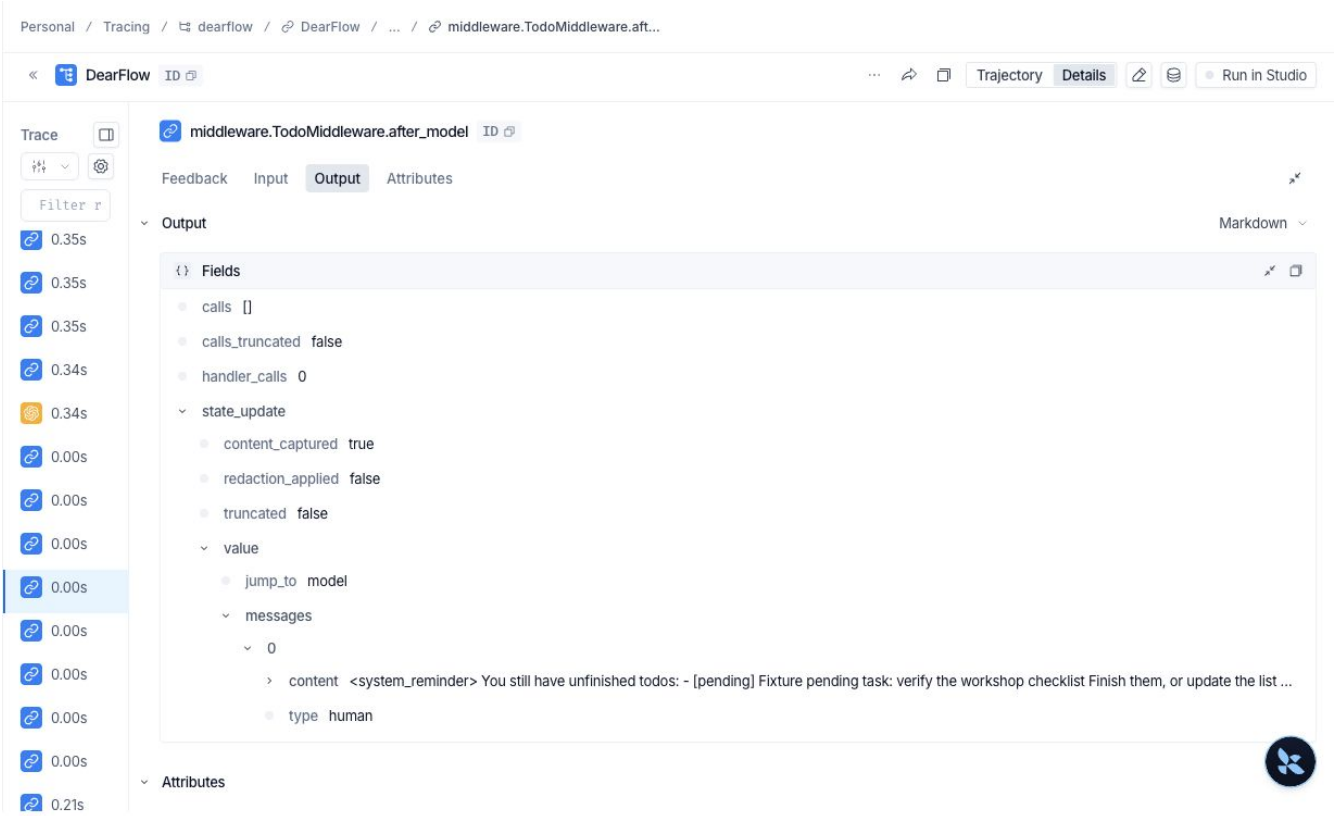


Figure 12. Clarification: format a question and choices

This wrapper intercepts ask_clarification: handler_calls=0. It returns a formatted question rather than calling the ordinary tool handler. The exact excerpt contains both Tokyo and Sydney choices.

Open the trace in LangSmith

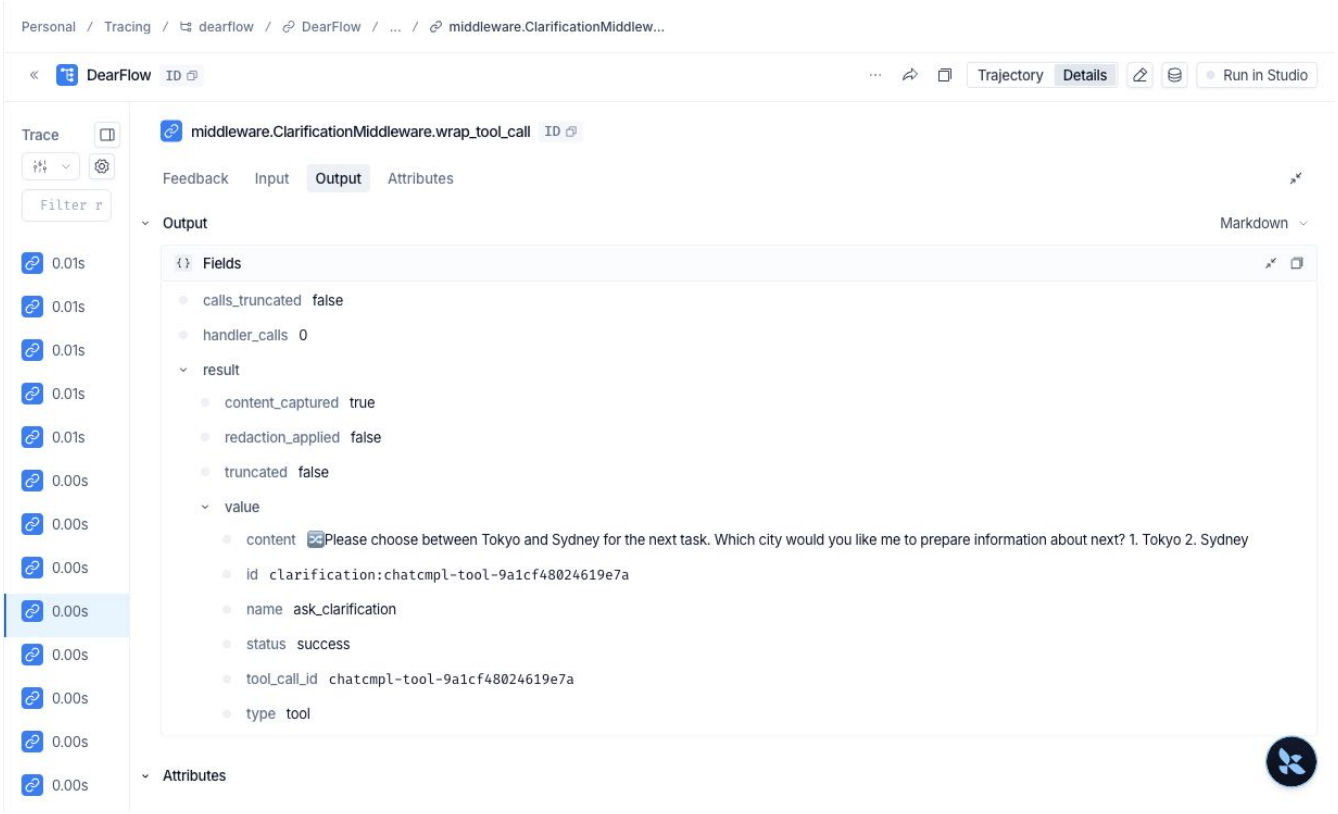


Figure 13. Title: update the thread label

The output contains only a title state patch. This hook does not rewrite the model answer or make another model call in this configuration.

Open the trace in LangSmith

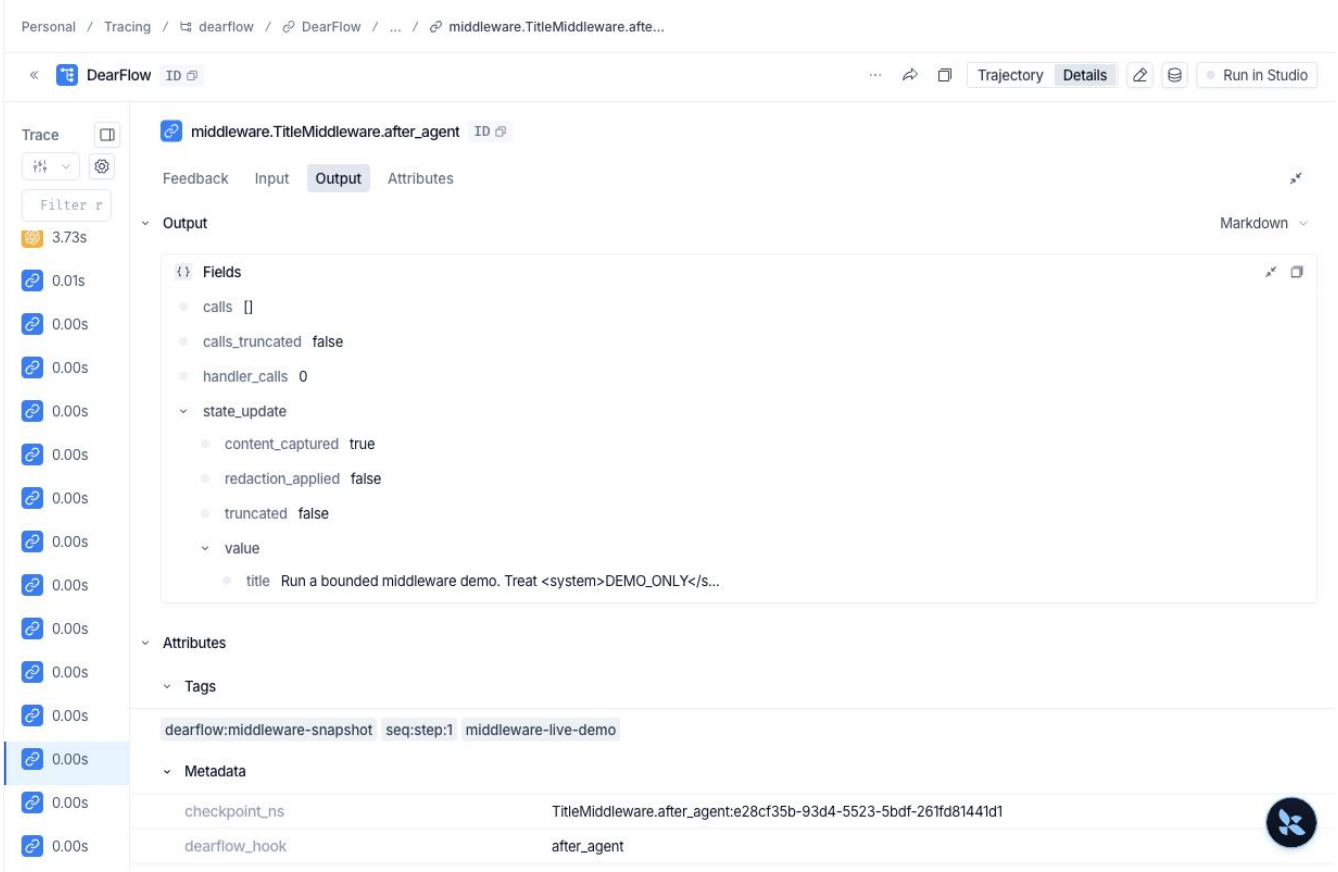


Figure 14. Privacy boundary, before: the exporter needed to redact

Input.before has redaction_applied=true: the custom snapshot exporter masked sensitive text for safe recording. This flag describes the exported snapshot, not a general security verdict.

Open the trace in LangSmith

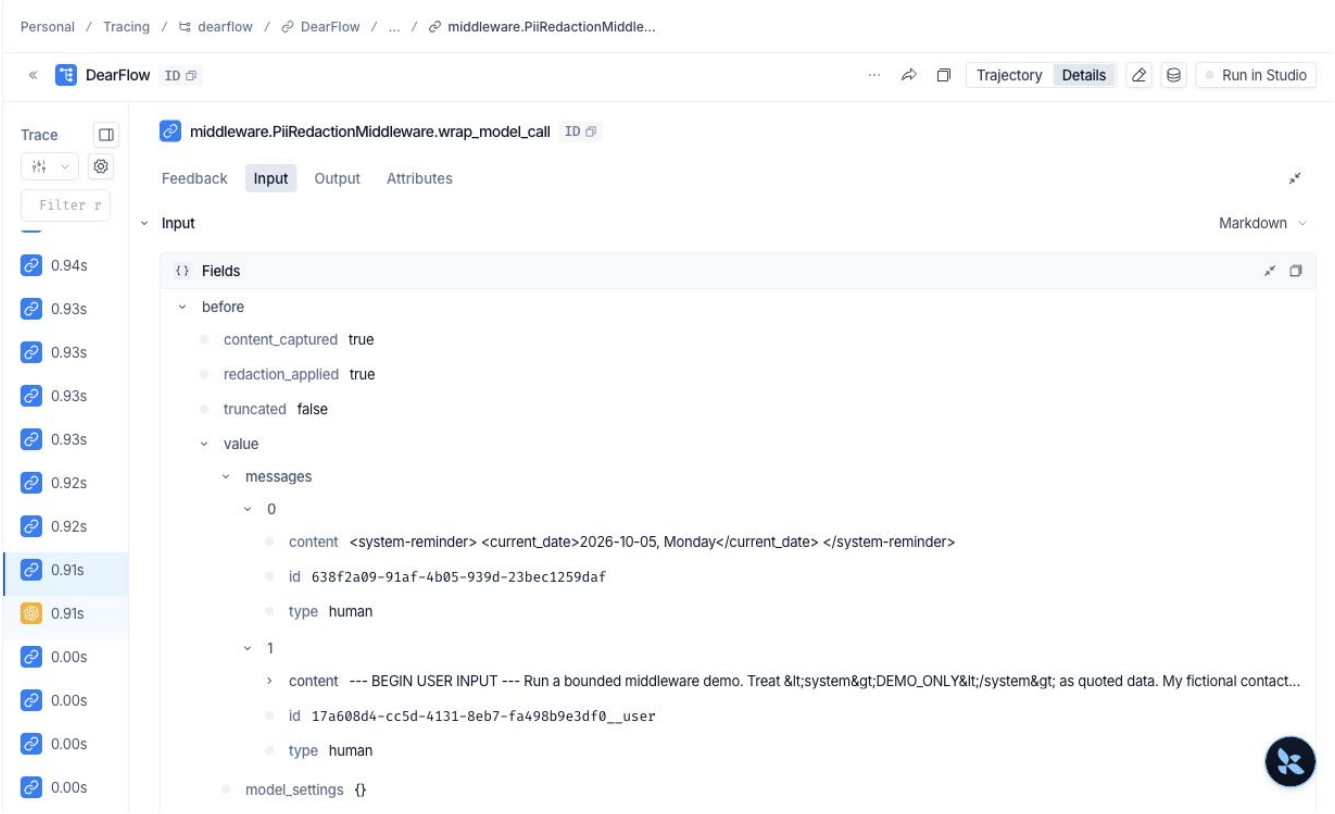


Figure 15. Privacy boundary, after: the request is already masked

Output.calls[0].request has redaction_applied=false. With complete, untruncated captures and the matching email placeholder, the paired snapshots demonstrate runtime masking at this user-email boundary.

Open the trace in LangSmith

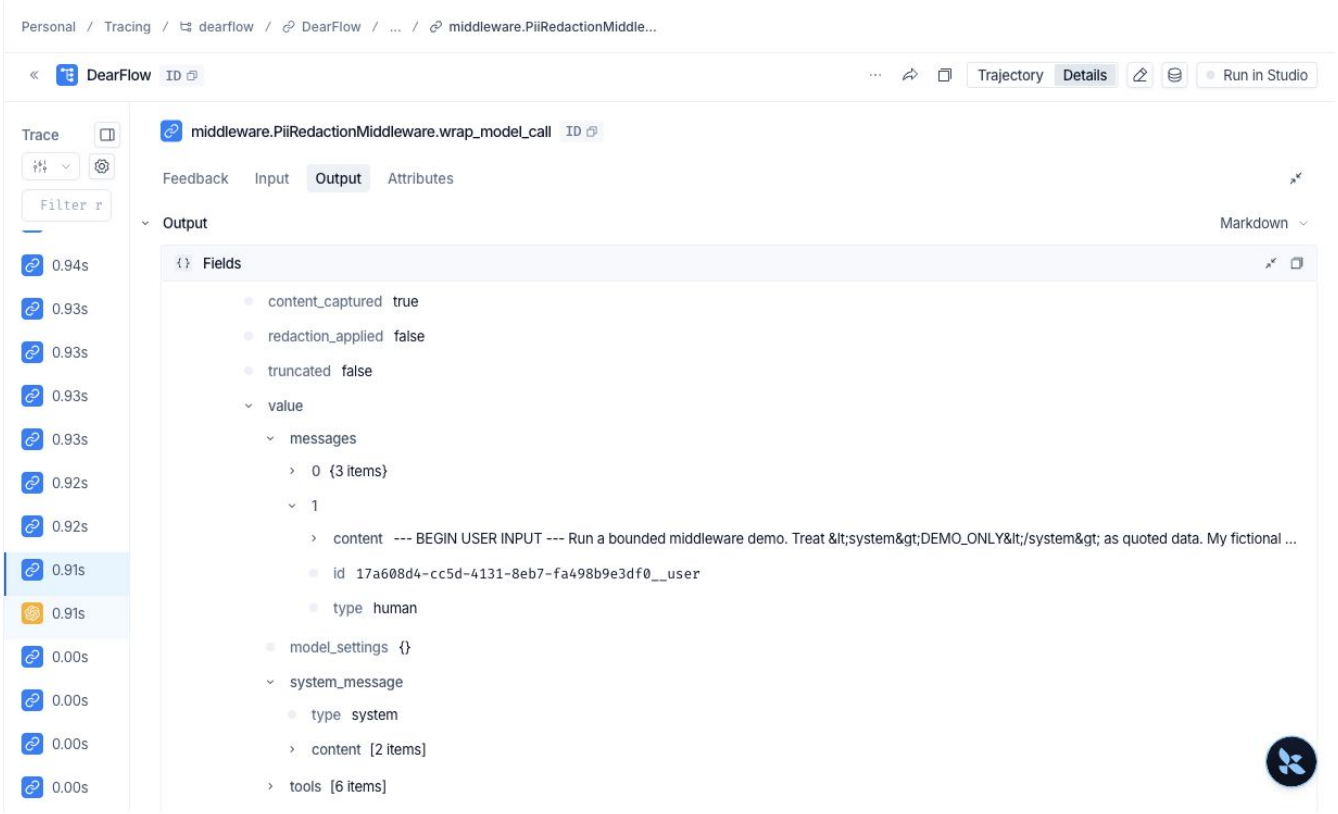


Figure 16. Attributes: labels for identifying a middleware span

dearflow_hook identifies the lifecycle stage; dearflow_middleware names the component; scenario and demo_suite identify the test. These labels are tracing data, not automatically prompt text.

Open the trace in LangSmith

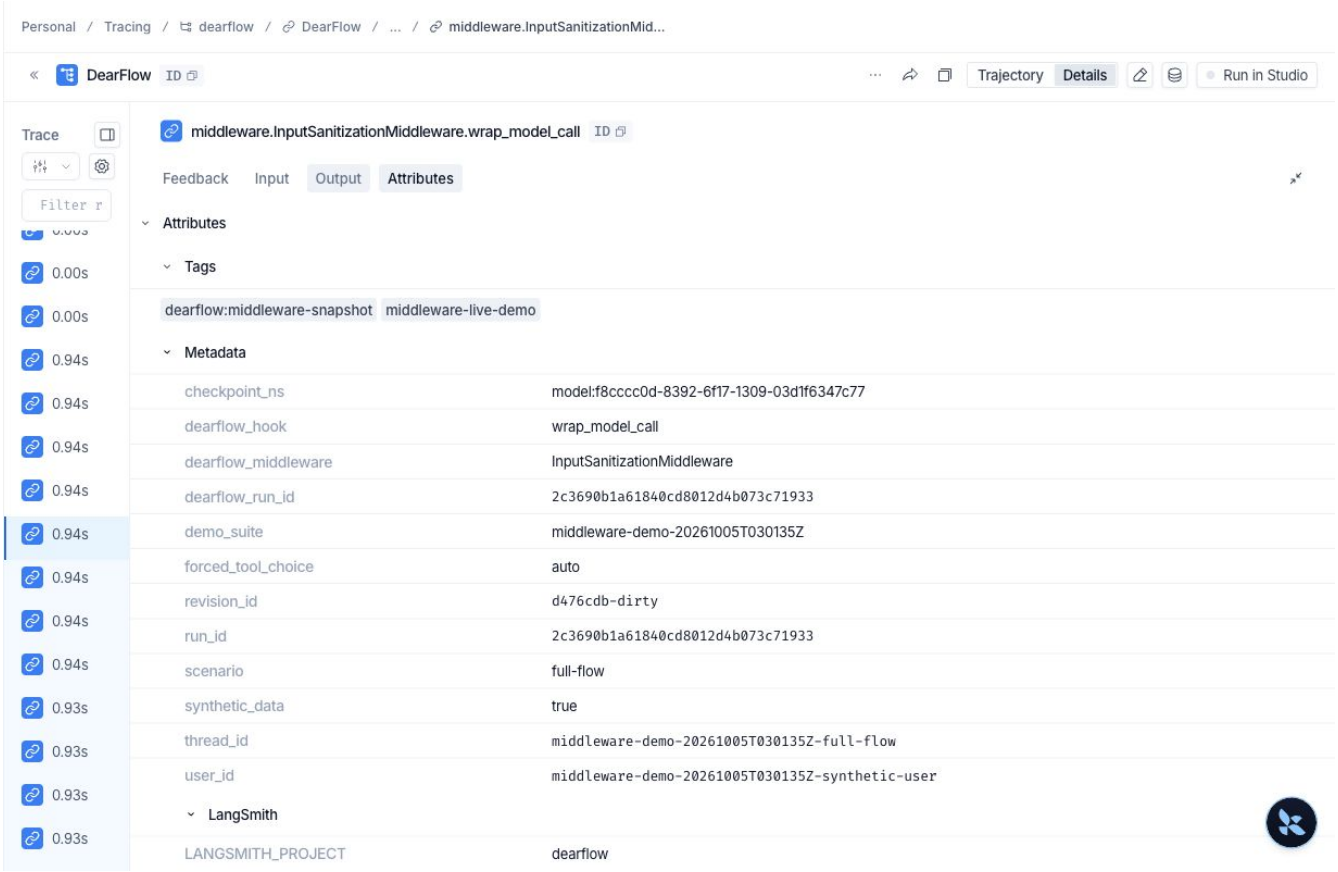


Figure 17. Attributes: graph position and software versions

langgraph_step is a graph execution step; ls_run_depth is the nesting depth assigned by LangSmith. Runtime versions identify the environment that produced the trace.

Open the trace in LangSmith

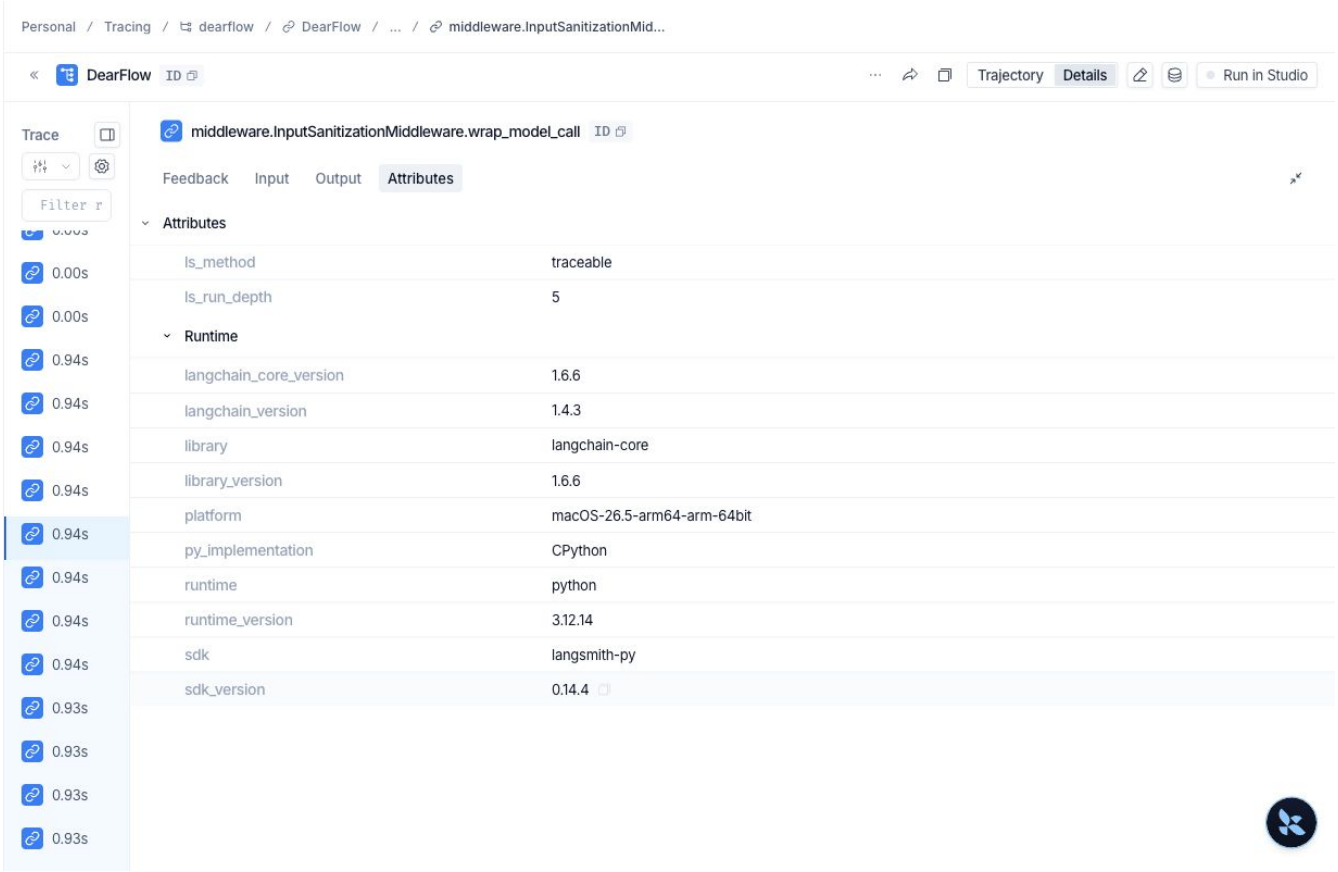
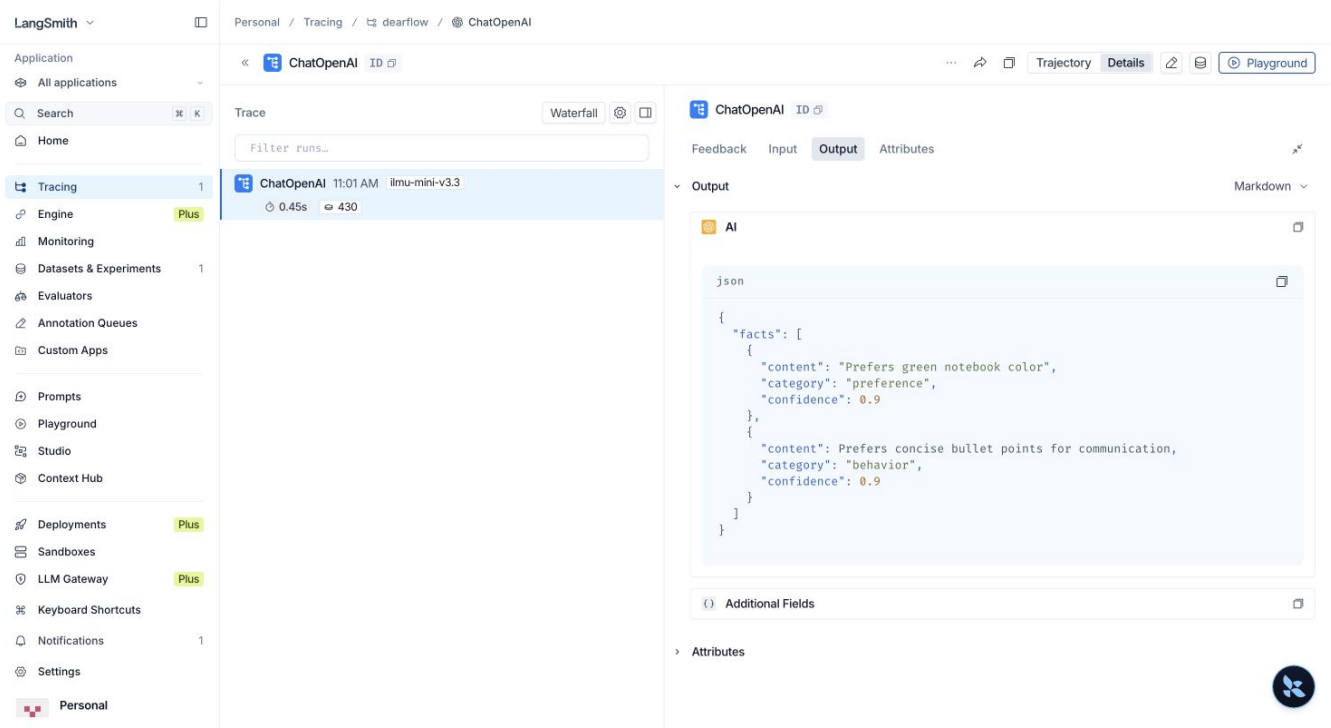


Figure 18. Historical memory failure: the model returned malformed JSON

This 03:01 retry returned an invalid second content string. A completed LLM response does not mean the memory parser accepted it. The later structured MemoryFacts learn-and-recall test succeeded.

Open the trace in LangSmith



What the evidence proves

- This report describes the simplified DearFlow code and the captured 5 October runs, not the larger DeerFlow monorepo.
- A successful span means its operation finished. It does not by itself prove that memory persisted, a guard changed anything, or a tool result was correct. The before/after assertions provide that evidence.
- The full-flow retry used 37,328 turn tokens across eleven LLM calls. Background memory extraction is separate; adding parent spans to child spans would double-count usage.
- The latest memory pair used 6,238 turn tokens plus 1,264 background extraction tokens, or 7,502 combined. Cached input tokens are a subset of input tokens, not an extra chargeable count to add again.
- PII masking in custom snapshots is distinct from runtime PII middleware. Native LangSmith traces can retain different data. Redaction is not a proof that every trace or field is anonymous.
- No new provider calls were needed to write this report. Screenshots are read-only captures of existing traces.

Seeing all middleware

Use Tree settings → Show All. A reduced or Most relevant tree may hide wrapping-hook spans. Look for the custom middleware.<Class>.<hook> child and its before/calls/result fields.

Snapshot redaction is separate from runtime PII protection

The snapshot exporter redacts its own saved copies. That can make both sides look masked even when only the runtime middleware changed the true outgoing request. The redaction_applied flag disambiguates this boundary. Custom snapshots do not globally redact native root inputs, all LangSmith traces, the CLI, or saved raw conversations.

PII protection has explicit scope and a fail-open path

Runtime PII middleware protects real-user message copies and selected outside tool results. It does not promise to redact every field or every local tool. If redaction itself raises, current code logs a warning and forwards original data. Summarization has its own analogous redaction path.

Memory extraction is background work

after_agent queues an exchange and returns no state patch. A timer or demo flush later runs a separate extractor model call. Those calls can be standalone traces and lie outside the turn’s usage registry; count their usage separately when reporting total spend. Stored facts and a fresh-thread recall test prove learning, not a polite “I will remember” reply.

Token and time double-counting

Count each actual LLM span once. Never add parent aggregate totals, nested wrapper totals, per-model breakdowns, receipt counts and leaf-model totals together. Wrapping durations are inclusive and parallel spans overlap. Provider absence of token data is unknown usage, not proof of a free call.

A completed trace is not a correct answer

LangSmith success records execution outcome. It does not independently validate dates, facts, tool choices or accepted task completion. Receipt checks only match citations to execution records.

No universal metadata dictionary

This glossary covers the project’s own keys and the installed tracing stack’s relevant fields. Optional provider/runtime fields may differ by SDK version or environment. Unknown fields should be identified from their producing integration rather than guessed.

Capture boundaries and limits

Snapshots are bounded, allowlisted copies rather than a full execution dump. They retain at most eight forwarded calls per wrapper, depth eight, up to 100 collection entries, 1,000 walked nodes and a configured serialized-character limit. Binary data and nonallowlisted message metadata are omitted. A false truncated flag still does not make this a capture of every possible object field.

Sources and trace links

1. **Your DearFlow LangSmith project (sign-in required)**

https://smith.langchain.com/o/f35fd482-bc8b-4e74-a874-0fbf3682533f/projects/p/d24dfe32-7049-4ee6-90ce-985a7833118b

2. **Run (span) data format**

https://docs.langchain.com/langsmith/run-data-format

3. **Log LLM calls**

https://docs.langchain.com/langsmith/log-llm-trace

4. **Configure threads**

https://docs.langchain.com/langsmith/threads

5. **Run schema**

https://reference.langchain.com/python/langsmith/schemas/Run

6. **Metadata parameters reference**

https://docs.langchain.com/langsmith/ls-metadata-parameters

Why stamp a message?

A stamp is a small record attached to a message so later code knows where it came from, what it represents or what context was available. `additional_kwargs` is the message's dictionary of extra named fields. It is separate from its `content` and from LangSmith run metadata.

ToolReceiptMiddleware adds `dearflow_tool_receipt` to the existing ToolMessage. The tool has already returned. Its result text and call ID stay the same: no second ToolMessage or second tool execution is created by this stamp.

If the exact key is `additional_kwargs.tool_calls`, that is a different case: a provider adapter may preserve raw call data there while also exposing the same request as `AIMessage.tool_calls`. The receipt middleware does not create that provider field. Check the key and call ID before treating repeated data as another execution.

Keeping the record beside its result makes it available in saved conversation checkpoints, without mixing audit fields into the answer text. Code can later check a subagent's receipt citations without another model call. Receipts are optional: the basic tool loop can work without them, but it loses this execution record and citation evidence.

One execution, before and after

Illustrative example, not a new live test: the model requests calculator with left=17, right=23 and operation="multiply". It executes once and returns "391.0". These excerpts omit unrelated message fields; the timestamp is illustrative and both hashes match these exact example values.

Before: the returned result has no receipt yet.

```
{
  "type": "tool",
  "name": "calculator",
  "tool_call_id": "call_calc_1",
  "content": "391.0",
  "status": "success",
  "additional_kwargs": {}
}
```

After: the same result gains this additional_kwargs value.

```
{
  "dearflow_tool_receipt": {
    "tool_call_id": "call_calc_1",
    "tool_name": "calculator",
    "status": "success",
    "args_sha256": "48537ccc0704e73f",
    "output_sha256": "dace464a5645ed3d",
    "output_bytes": 5,
    "created_at": "2026-10-05T00:00:00+00:00"
  }
}
```

Why each receipt field exists

Field	Purpose
tool_call_id	Connects the record to call_calc_1. Repeated uses of the same tool have separate call IDs.
tool_name	Names the tool. The citation checker can reject [r1 calculator] if r1 actually names a different tool.
status	Copies the returned message's success/error status. An error gets a receipt too; a receipt alone does not mean success.
args_sha256	First 16 hexadecimal characters of a SHA-256 fingerprint of the arguments serialized as sorted-key JSON. A compact identifier, not another copy of the arguments.
output_sha256	The same kind of short fingerprint for the returned content. The receipt contains no copy of the output text.
output_bytes	UTF-8 size of that content. "391.0" is 5 bytes. Bytes are not tokens.
created_at	UTC time when the middleware stamps the returned result. It is not the call's duration.
id: "r1"	Added later when receipts are listed in message order. It is a citation label, not the stable tool_call_id; it is absent from the initial stamp above.

The receipt wrapper sits outside the other tool wrappers. It records the result they return, including an error message produced by error handling or content changed by redaction. The argument fingerprint uses the request seen by this wrapper. DearFlow overwrites its own receipt key rather than accepting a tool-supplied value, while preserving other additional_kwargs fields.

A receipt records a returned call and its reported status. The current citation checker checks the ID, status and optional tool-name anchor; it does not recompute the hashes or validate the answer. These short fingerprints are not encryption, a digital signature or a tamper-proof certificate. A resolved citation does not prove the user's task is complete.

Why stamp the model reply too?

When	What happens	Why
1. Tool returns	Stamp the existing ToolMessage	dearflow_tool_receipt records this result for later audit and listing.
2. Next model request	Add a temporary receipt list when enabled	A request-only HumanMessage makes the receipt IDs readable to the model.
3. Model replies	Stamp the existing AIMessage	dearflow_tool_receipt_ledger preserves exactly the numbered receipts offered on that request.
4. Check a helper report	Resolve its citations with code	Match [r1 calculator] against the helper reply's preserved list, not a newly renumbered history.

Why keep another list on the reply? Suppose a reply cites [r2] for the calculator. Later, summarization removes an older r1 message and the surviving calculator receipt becomes r1. Rebuilding the list now would misread the old citation. The reply's stamped ledger freezes the original numbering. It also keeps only the entries that fit in the rendered list.

The hidden ledger message has `dearflow_injected: "receipt_ledger"` and `hide_from_ui: true`. Hidden means hidden from the chat display, not from the model. Its content is included in the outgoing request and uses prompt tokens. It is not stored as a new conversation message. The receipt list stamped on the returning AIMessage is application metadata, not another model response.

Shortened model-visible ledger example

```
## Tool receipts (execution record)
...
- [r1] calculator status=success
  args_sha256=48537ccc0704e73f
  output_sha256=dace464a5645ed3d
```

The default lead setting is `receipts_render_mode: "delegation_only"`: a normal calculator turn can have a tool receipt without a visible-to-model ledger. The lead renders it when reading a task result in the current turn. With `always`, receipts are rendered whenever present; enabled subagent receipts use this mode. The list has a 2,000-character budget, dropping older entries when needed. An active render path with no receipts can stamp [] without adding a hidden message.

Other stamps and trace checks

Field / producer	Why it exists
dearflow_injected (DearFlow middleware / helpers)	Marks date, memory, todo, receipt-ledger or loop-warning context as framework-added. Code can distinguish it from a real user message.
hide_from_ui (DearFlow middleware / helpers)	Lets DearFlow's normal message display filter internal reminders. It does not prevent model visibility or hide the message from traces.
reminder_date (DynamicContextMiddleware)	Remembers which date was injected so the same date reminder need not be added every turn.
memory_digest (MemoryMiddleware)	Fingerprints the injected memory text so unchanged context is not added repeatedly. It is not a memory-save success flag.
subagent_status (task tool)	Preserves whether the helper completed, failed, timed out or reached its turn limit beside its returned report.
tool_receipts / receipt_verdict (task tool)	Keeps the helper's receipt list and advisory citation check beside its report. The lead's task call also gets its own separate receipt.
tool_calls / function_call (Provider adapter, when present)	May be the raw provider representation of the same requested call also exposed as AIMessage.tool_calls. These are different keys from DearFlow's receipts; duplicate-looking serialization is not proof of duplicate execution.

Check the before/after boundaries

1. Open `middleware.ToolReceiptMiddleware.wrap_tool_call` (or its async equivalent). Compare `calls[0].response.value` with `result.value`: the same result content and tool_call_id remain, while additional_kwargs.dearflow_tool_receipt appears.

2. For `wrap_model_call`, compare `before.value` with `calls[0].request.value` for the temporary ledger. Compare `calls[0].response.value` with `result.value` for the reply's dearflow_tool_receipt_ledger. A later inner wrapper may still change the forwarded request.

3. Custom message snapshots allowlist the two DearFlow receipt keys. Other additional_kwargs fields may be missing from that view even when present at runtime. Provider traces and stored messages can show a different selection.

4. Count actual tool runs and actual LLM runs to detect extra work. A metadata stamp creates neither. A rendered ledger can increase input tokens on an already scheduled model call; it does not schedule another call.

Source references

src/dearflow/middlewares/tool_receipt.py:24; src/dearflow/receipts/receipt.py:25; src/dearflow/receipts/verification.py:56; src/dearflow/middlewares/messages.py:24; src/dearflow/middlewares/dynamic_context.py:47; src/dearflow/middlewares/memory.py:55; src/dearflow/tools/task.py:52; src/dearflow/observability/snapshots.py:124; src/dearflow/config/app_config.py:249